

1 Introduction 初稿

第 1 段：背景

将大语言模型用作知识图谱与结构化数据的推理器，已成为一条重要路线。已有工作通过关系路径规划、图上搜索、子图检索、工具调用或受约束解码等方式，使模型能够在多跳问答与图任务中利用显式图结构，并报告了可观的性能提升 [RoG; SoG; Graph-CoT; GraphWalk; FiDeLiS]。然而，这些方法共享一个基本前提：图结构需要以某种形式进入自回归语言模型的 token 上下文。由于图本身没有天然线性顺序，节点与边通常必须被序列化为 JSON、邻接表、边列表或自然语言描述。于是，一个本应由结构关系决定的推理任务，在输入端就被附加了一层与结构无关的顺序与位置信息 ## 第 2 段：缺口

这层位置信息并非无害。已有研究反复表明，LLM 对输入顺序和序列化方式高度敏感：长上下文中模型可能偏好首尾位置而忽略中段 [Lost in the Middle]，多选题中调换选项顺序即可显著改变预测 [MCQ-order]，而在图推理中，仅改变图描述格式、节点命名或边排列顺序，也可能影响模型表现 [Lost in Serialization]。这暴露出一个更细的诊断问题：当模型在图任务上答对，或者答案随着图干预而改变时，它究竟是在遵循图结构，还是在利用序列化后某个节点，尤其是正确终点，所处的位置？换言之，答案随图改变只是图忠实性的必要不充分条件：真正遵循图结构的模型应当对结构干预敏感，但结构干预下的答案变化，也可能来自终点位置等文本线索的同步变化。若不控制这一混淆，原始图干预敏感性可能系统性高估模型的图推理能力 ## 第 3 段：CoT faithfulness 过渡

这一问题与已有 CoT faithfulness 研究形成方法论上的呼应。早期工作区分了 plausibility 与 faithfulness，指出推理过程看起来合理并不意味着它真实反映模型决策依据 [Jacovi]。随后，行为干预式 CoT faithfulness 研究表明，模型可能被输入中的偏置信号系统性影响，却不会在生成的思维链中承认这些因素 [Turpin; Lanham]。跨模型家族研究进一步显示，依赖模型是否口头承认偏置信号来判断忠实性本身并不可靠 [Young]。本文继承这种干预式视角，但将忠实性的对象从语言解释转向外部图结构：不问模型是否在 CoT 中承认真实原因，而是问模型答案和路径是否受序列化图结构因果控制 ## 第 4 段：方法

为此，本文提出 Graph-Intervention Faithfulness，简称 GIF。GIF 包含两层受控干预：首先，我们构造正确终点不同且经符号验证的反事实图对，用于测量模型答案是否随图结构改变；其次，我们在每个图内部生成多种位置控制序列化，包括 endpoint-first/middle/last 和 decoy-last，用于检验这种表面图敏感性是否仍然存在。基于此，定义 Raw Graph Intervention Sensitivity、Position-Controlled GIS 和 Graph Faithfulness Inflation，其中 $GFI = Raw\ GIS - PC-GIS$ ，用于量化未控制位置时被高估的图忠实性；同时使用 Endpoint Anchoring Rate 衡量模型是否倾向于选择最后出现或最显著的节点。在路径输出设置下，我们进一步以 Path Validity(每条边是否合法)与 Trace-Answer Consistency 区分“轨迹自治”与“图上合法”，二者的脱钩正是深层失败的诊断信号。GIF 是黑箱、任务无关的，并不依赖模型自述或自然语言解释，而是直接检验结构干预与模型行为之间的关系。遵循忠实性研究中的灰度观点 [Jacovi]，GIF 并不声称证明模型“真正会图推理”，而是量化特定设置下位置启发式对模型行为的主导程度。## 第 5 段：验证

第 5 段：验证

我们在符号图上对该框架进行了验证。实验刻意使用无语义先验的随机符号节点，以隔离结构追踪能力，并通过 prior-only 与 candidate-only controls 排除答案先验和候选偏好造成的混淆。实验覆盖两类按诊断目的设计的图族：链式图用于暴露 answer-only 接口下的 endpoint-position shortcut，分叉图用于检验模型是否能在多跳分支结构中持续维护当前状态。我们评估 DeepSeek、Qwen 与 GPT-5.4-mini 等模型，并比较 no-thinking、thinking 与 verifier-retry 等推理设置。

实验揭示出两类图推理假阳性，且其主导形态随任务难度和输出接口发生切换。在链式图的弱提示设置下，模型表现出明显的答案层图跟随虚胖。例如在 chain-4hop + direct_minimal 中，DeepSeek-V4-Flash 的 Raw GIS 达 68.5%，但 PC-GIS 降为 0%，GFI 为 68.5%，decoy-last EAR 达 63.8%；Qwen Max 也呈现相同方向，Raw GIS 为 38.0%，PC-GIS 仍为 0%。进一步的 chain 长度扫描显示，DeepSeek-V4-Flash 稳定依赖 endpoint-position shortcut，GPT-5.4-mini 表现为中间型捷径依赖，而 Qwen Max 在较长链上逐渐失去图干预敏感性。这说明 Raw GIS alone 会高估图跟随能力，并掩盖不同模型之间性质不同的答案层失败。

当任务转入 branching-12hop 且使用 jsoncot_strict 结构化输出后，位置捷径不再是主要失败来源，另一种更隐蔽的路径层失败浮现：模型生成的路径常常与最终答案高度自治，却并不是图上的合法路径。DeepSeek-V4-Flash 的 Trace-Answer

Consistency 达 99.3%，但 PathGoldExact 仅为 48.5%，非法路径差距 Δ_{illegal} 为 50.8%；Qwen Max 的现象更极端，TAC 达 99.2%，PathGoldExact 仅为 8.4%， Δ_{illegal} 高达 90.8%。这表明，结构化 CoT 能让模型生成看似完整、答案一致的轨迹，却不能保证轨迹中的每一步都忠实于输入图边。

进一步地，thinking 与 verifier-retry 实验揭示了图跟随能力的可恢复性边界。DeepSeek-V4-Pro thinking 在 branching-12hop 上几乎解决任务，PathGoldExact 达 99.9%，但平均延迟为 46.7 秒；相比之下，DeepSeek-V4-Flash verifier-retry 从 pass@1 的 48.6% 提升到 pass@5 的 70.6%，累计延迟为 4.42 秒。Qwen Max 也从 8.4% 提升到 33.4%，但最终上限明显更低。说明外部符号验证能以较低延迟放大已有的图跟随能力，但不能凭空创造这种能力；其收益强烈依赖底座模型本身的状态追踪能力。## 第 6 段：贡献

本文贡献如下：

第一，问题层面，我们指出图任务中的高准确率和原始图干预敏感性都不足以证明模型忠实使用图结构；在序列化图推理中，答案随图改变可能被终点位置等文本线索虚高。

第二，方法层面，我们提出 GIF，一种位置控制的图干预忠实性诊断框架，通过反事实双图、四种终点位置控制和符号验证的唯一 gold path，区分真实结构敏感性与位置锚定效应，并定义 GFI 与 EAR 等指标。

第三，发现层面，我们揭示了 LLM 图推理中的两类假忠实性及其随任务难度的主导机制切换。弱提示链式图主要暴露答案层位置虚胖：模型答案看似随图干预改变，但位置控制后崩塌；困难分叉图则暴露路径层非法轨迹：模型生成的路径与答案高度自洽，却包含图中不存在的边。我们进一步表明，thinking 模式几乎消除这两类失败但代价较高，而 verifier-retry 能以较低延迟部分修复非法路径；不过其收益受底座模型能力限制，更多是放大已有图跟随能力，而非创造新的图推理能力。

2 Related Work 相关工作 初稿

2.1 语言化推理的忠实性（缩短）

思维链提示让语言模型在数学、逻辑和多步推理任务上更具能力，但越来越多的研究开始质疑：模型生成的推理过程是否真实反映了驱动其预测的真正因素。这一领域的一个关键区分是**合理性 (plausibility)** 与**忠实性 (faithfulness)**：一个解释可能对人类而言可读、可信，却仍然没有描述模型真实的决策过程 [Jacovi & Goldberg, 2020]。这一区分对本文的设定同样核心：一条图推理轨迹可能看起来结构连贯，但这并不意味着该轨迹忠实于输入图

已有工作表明，语言模型可能在事后为受偏置或捷径驱动的答案编造合理解释。Turpin 等人发现，模型会被诸如答案位置偏置或被暗示的答案等虚假线索影响，而其思维链解释往往略去这些线索，转而提供看似合理的替代理由 [Turpin, 2023]。另一些工作则通过扰动、删除或改写推理过程，观察模型答案是否相应改变，提出了对思维链忠实性的定量检验 [Lanham, 2023]。这些研究共同确立了一点：最终答案的正确性、推理的合理性与忠实性，是三个不同的属性

然而，现有忠实性工作大多关注**语言化**的推理是否忠实地报告了模型答案的成因。与之不同，本文研究的是另一种忠实性：模型的**结构化输出**是否忠实于一张显式的输入图。在图推理中，真正相关的问题不只是模型能否给出一个合理的解释，而是它的答案与轨迹是否由合法的图结构、而非文本捷径所因果控制

2.2 行为干预与反事实诊断

由于模型的内部推理无法直接观测，许多忠实性研究依赖**行为干预**。这类方法不去问模型是否声称用了某个因素，而是对输入或推理过程施加干预，观察模型的预测是否如预期般改变。Turpin 等人的偏置注入实验正是这一范式的典型：关键证据并非来自单独阅读解释，而是来自比较模型在有偏置与无偏置上下文下的行为差异 [Turpin, 2023]。类似地，思维链扰动方法通过修改中间推理、观察最终答案是否变化，从而把不可观测的推理过程转化为可度量的行为敏感性 [Lanham, 2023]

反事实基准则通过**移除替代捷径**来延伸这一逻辑。例如 CofCA 构造反事实多跳问答样本，以降低模型对记忆性事实关联的依赖，检验模型能否真正沿给定上下文完成多步推理 [CofCA, ICLR 2025]。这与本文的动机高度相关：CofCA 针对的是**记忆造成的推理虚胖**，而本文框架针对的是**序列化造成的图跟随虚胖**；两种设定都在追问：当某条捷径被系统性地控制后，表面上的推理表现是否还能存留

近期工作还进一步区分了语言化忠实性与更广义的因果/结构忠实性。Young 等人检验开源推理模型的思维链是否显式承认了影响答案的提示或偏置，但这仍然是在检验模型是否**报告**了相关因素 [Young, 2026]。与之相对，本文追问的是模型输出是否

在**结构上**忠实于图本身。诸如计算图验证之类的白盒方法, 通过检查内部电路或归因结构提供了一条互补路径, 但它们依赖特定模型、且需要访问内部激活 [CRV, 2026]。GIF 则是一个**黑箱**诊断框架: 它对图结构、序列化位置和路径合法性施加干预, 评估模型可观测的答案与轨迹, 是否表现得与“忠实于图的推理”相一致

2.3 图推理与结构化轨迹忠实性

越来越多的工作用图、知识图谱或基于工具的图导航来增强 LLM 推理。一类方法依赖**显式的关系路径或知识图谱检索**: 它们让 LLM 沿检索到的关系路径或受约束解码的知识图谱路径作答, 以提升多跳问答的事实性与可解释性 [RoG 2024; Follow-the-Path 2026; Graph-CoT 2024; FiDeLiS 2025]。另一类则把图推理建模为**迭代式图导航或工具调用**: 模型在图上逐步搜索下一跳、或通过工具接口遍历结构 [SoG 2025; GraphWalk 2026; GCR 2025]。两类方法都证明了显式图结构能提升表现, 但它们往往依赖同一个隐含假设: 只要提供、生成或把一条路径用作上下文, 模型就是在沿着这条路径推理 **

而这正是本文要挑战的假设。**路径出现在输入或输出中, 本身并不意味着模型因果地遵循了这条路径**。模型可能输出一条与其最终答案完全自治的轨迹, 而这条轨迹在底层图上却是非法的; 换言之, **轨迹—答案一致性并不等于图上合法性**。这一区分对那些把“显式路径生成”当作训练或评测前提的方法尤为重要。例如, 路径奖励方法必须先强制模型外显图路径, 才能计算与路径对齐的奖励 [KG-reward, 2026]。在我们的 pilot 实验中, 强制显式输出路径将位置锚定从近乎完全 (GFI $\approx$ 100%) 压低至边缘水平 ($\approx$ 5%)。这提示, 依赖显式路径外显的方法 (如路径奖励训练) 其有效性可能部分来自这一**被忽视的接口效应**, 而该效应从未被当作一个需要验证的忠实性条件。看似只是工程细节的东西, 可能恰恰是控制图推理是否忠实的隐藏变量。

近期一些图推理评测已经开始质疑: 正确的答案是否真的由合法路径支撑。例如 FidelityAcc 仅当答案正确**且**预测路径可达时, 才将一次预测计为正确, 从而揭示出“高答案准确率掩盖非法推理路径”的情况 [FidelityAcc, 2024]。这是超越纯最终答案评测的重要一步。但这类诊断止步于**路径可达性**, 并未隔离序列化位置捷径。GIF 在此基础上延伸, **同时检验答案层的位置锚定与路径层的轨迹合法性**: 既问答案是否受图干预控制, 也问生成的轨迹是否真的是图上的一次合法游走。

序列化敏感性提供了另一条紧密相关的动机。由于图是无序结构、而 LLM 处理的是序列, 图输入在被处理前必须先被线性化。已有工作表明, LLM 的图推理可能对边的顺序、节点命名、图描述格式等序列化选择敏感 [Lost in Serialization, 2026]。类似的位置效应也出现在列表式重排、推荐、多选题问答和长上下文推理中——本应与语义无关的排序选择, 却会影响模型预测 [InvariRank 2026; RISE 2025; 选项顺序 2024]。这些发现表明, 即便任务语义本应对排列不变, LLM 仍可能把序列位置当作决策线索。然而, 序列化研究通常只关注重排输入后**答案**的变化, 并未区分两种不同的失败模式: **答案层的位置锚定, 与路径层的非法轨迹生成**。模型可能回答最后出现的那个终点, 也可能输出一条看起来连贯、但其节点与边并不对应图中任何合法游走的路径。GIF 正是为**同时诊断这两种失败**而设计: Raw GIS、PC-GIS 和 GFI 衡量图干预是否在终点位置之外真正控制了答案, 而路径合法性、轨迹—答案一致性、路径精确匹配 (Path Gold Exact) 和失败跳数分析, 则检验结构化轨迹是否为图上的合法路径。

最后, GoV、GraphReason、GNNVerifier 等基于图的验证方法也凸显了结构感知评测的重要性 [GoV 2026; GraphReason 2024; GNNVerifier 2026]。这些工作验证生成的推理结构、任务规划或多路径推理图, 表明图结构能暴露整体式 LLM 评判所遗漏的失败。然而, 它们通常并不检验: 呈现给 LLM 的图表示是否对序列化顺序稳健, 也不检验轨迹—答案一致性是否就意味着相对外部输入图的合法性。本文与这些方法形成互补——我们把**图忠实性本身**当作诊断对象: 不只是问一张生成的推理图看起来是否合法, 而是问模型的答案与轨迹是否因果地扎根于所提供的图结构。

总体而言, 已有工作分别证明了: 语言化推理可能不忠实、图增强提示能提升性能、LLM 对序列化顺序敏感。但此前工作尚未系统地区分图推理中的两种不同失败: **答案层的位置锚定与路径层的非法轨迹生成**。GIF 通过结合图干预、位置控制变体与路径级合法性诊断, **同时覆盖这两个层面**。

总体而言, 已有工作分别回答了三个相邻但不同的问题: 图推理工作主要关注给模型提供图结构是否能提高最终答案表现; 图序列化工作关注不同线性化方式、节点顺序或边排列是否会影响模型输出; CoT faithfulness 工作关注模型生成的语言化解释是否忠实反映其答案来源。GIF 关注的是另一个更细的诊断问题: 在图被序列化为文本之后, 模型的答案和结构化轨迹是否仍然受到输入图结构本身的约束, 而不是由终点位置、边列表顺序或输出内部自治性伪装出来。为此, GIF 将反事实图干预、位置控制序列化和路径级合法性验证结合起来, 同时诊断答案层的图跟随虚胖与路径层的自治但非法轨迹。

3 Graph-Intervention Faithfulness

本节提出 Graph-Intervention Faithfulness, 简称 GIF。GIF 是一个黑箱诊断框架, 用于检验大语言模型在序列化图任务中的行为是否真正受输入图结构约束, 而不是由文本位置线索、答案先验或输出内部自洽性伪装出来。

GIF 关注两层忠实性。第一是答案层忠实性: 模型答案是否随着图结构干预而改变, 并且这种变化是否能在控制终点位置后仍然保持。第二是路径层忠实性: 当模型输出结构化路径时, 该路径是否真的是输入图上的合法路径, 而不只是与最终答案自洽。

核心思想可以概括为两句话:

答案随图干预改变; $\nRightarrow$; 答案忠实遵循图,

轨迹与答案一致; $\nRightarrow$; 轨迹在图上合法.

因此, GIF 同时结合反事实图干预、位置控制序列化、路径级合法性验证和符号 verifier-retry, 来诊断模型是否真正忠实于输入图结构。

3.1 Task and Counterfactual Graph Pairs

给定有向图

$$G = (V, E),$$

其中 (V) 为节点集合, $E \subseteq V \times V$ 为有向边集合。本文关注一种受控的定长多跳图推理任务: 给定起点 $s \in V$ 和跳数 (k) , 模型需要从 (s) 出发, 沿图中合法有向边恰好走 (k) 跳, 并输出最终到达的节点。

一条长度为 (k) 的合法路径记为:

$$p^* = (v_0, v_1, \dots, v_k),$$

其中

$$v_0 = s, \quad v_k = y,$$

并且

$$\forall i \in 0, \dots, k-1, \quad (v_i, v_{i+1}) \in E.$$

任务的标准答案为路径终点:

$$y = v_k.$$

语言模型并不直接处理抽象图 (G) , 而是处理图的线性化文本表示。因此, 我们定义一个序列化函数:

$$\sigma_\alpha(G) = (x_1, x_2, \dots, x_n),$$

其中 α 表示一种具体的序列化配置, 包括节点标号、边顺序、语法格式和位置控制策略。给定任务描述 (T) 和序列化图 $(\sigma_\alpha(G))$, 模型输出最终答案 $\hat{a}$, 并在结构化提示下额外输出显式路径 $\hat{p}$:

$$f_{\theta}(T, \sigma_{\alpha}(G)) \rightarrow (\hat{p}, \hat{a}).$$

在 answer-only prompt 下，模型只输出答案，可视为：

$$\hat{p} = \emptyset.$$

为了检验模型答案是否真正受图结构控制，我们构造反事实图对：

$$(G_1, G_2).$$

两张图共享相同的起点 (s) 和跳数 (k), 但对应的合法 (k)-hop 终点不同：

$$G_1 : s \xrightarrow{k \text{ hops}} y_1,$$

$$G_2 : s \xrightarrow{k \text{ hops}} y_2,$$

且

$$y_1 \neq y_2.$$

对应的黄金路径分别为：

$$p_1^* = (s, \dots, y_1), \quad p_2^* = (s, \dots, y_2).$$

如果模型真正遵循输入图结构，那么当图从 (G_1) 被干预为 (G_2) 时，模型答案也应该从 (y_1) 改为 (y_2)。这构成答案层图忠实性的一个必要条件。

不过，答案随图变化并不充分。模型可能并没有沿图推理，而只是利用了与图干预同步变化的文本线索，例如正确终点总是出现在边列表最后。GIF 的后续位置控制正是为了排除这类混淆。

在实验构造中，我们使用随机符号节点以削弱语义先验，并通过 prior-only 和 candidate-only prior controls 检查无图条件下的固定答案偏好。若模型在无图条件下已经稳定输出某一候选答案，则该样本可能被视为 prior-confounded，并在主实验中剔除或单独报告。本文的主实验中，prior controls 表明答案先验和候选偏好不能解释主要结果。

此外，本文核心实验采用唯一黄金路径设定：给定 $((G, s, k))$ ，从 (s) 出发恰好 (k) 跳可达的合法终点唯一，对应黄金路径 (p^*) 也唯一。该唯一性通过符号枚举验证。需要强调的是，唯一性主要用于定义 *PathGoldExact*；路径合法性、*TAC* 和 *FailureHop* 并不依赖唯一黄金路径。若未来扩展到多合法路径场景，可将单一路径 (p^*) 替换为黄金路径集合 (P^*)。

3.2 Position-Controlled Serialization

反事实图对检验模型答案是否随图改变，但无法排除序列化位置捷径。尤其是在边列表或邻接表序列化中，正确终点可能位于文本中的显著位置，例如最后一条边、最后一个节点或列表末端。此时，模型即使不沿图推理，也可能通过“选择最后出现的节点”获得较高的图干预敏感性。

为此，GIF 对同一张图 (G) 构造一组位置控制序列化：

$$\Sigma(G) = \sigma_{\text{first}}(G), \sigma_{\text{middle}}(G), \sigma_{\text{last}}(G), \sigma_{\text{decoy}}(G).$$

其中, σ_{first} 、 σ_{middle} 和 σ_{last} 分别将正确终点节点置于序列化边列表的较前、中间和结尾位置。这三个变体构成终点位置扫描, 用于检测模型是否稳定依赖正确终点的文本位置。

σ_{decoy} 则是一个对抗探针: 它将一个非答案诱饵节点放在结尾位置。该诱饵节点 (d) 满足:

$$d \neq y,$$

并且不是从起点 (s) 出发恰好 (k) 跳可达的合法答案。若模型在 decoy-last 条件下输出诱饵节点, 则说明模型可能依赖 “最后出现节点” 这一位置线索, 而不是沿图求解。

位置控制的目标是在保持图结构不变的前提下, 只改变文本位置线索。若模型真正忠实于图结构, 则对于任意位置控制序列化 ($\sigma_\alpha(G)$ in $\Sigma(G)$), 模型都应输出同一个图结构答案 (y)。

3.3 Answer-Level Metrics: Raw GIS, PC-GIS, GFI, and EAR

答案层指标用于诊断模型答案是否真正受图结构干预控制, 以及这种表面图敏感性是否被终点位置线索虚高。

对第 (j) 个反事实图对 ($(G_1^{\{j\}}, G_2^{\{j\}})$), 其对应终点为 ($y_1^{\{j\}}$) 和 ($y_2^{\{j\}}$)。在某一序列化条件 α 下, 定义模型是否随图干预正确改变答案:

$$C_\alpha^{(j)} \mathbb{1} \left[f_\theta(T, \sigma_\alpha(G_1^{(j)})) = y_1^{(j)}; \wedge; f_\theta(T, \sigma_\alpha(G_2^{(j)})) = y_2^{(j)} \right].$$

Raw Graph Intervention Sensitivity

Raw Graph Intervention Sensitivity, 简称 Raw GIS, 衡量模型在未进行位置控制或默认有利序列化条件下, 答案是否随反事实图干预改变。在本文实验中, raw 条件对应 endpoint-last 风格序列化, 即正确终点位于较显著的结尾位置。该设置用于刻画不控制位置时可能得到的表面图跟随表现。

对样本 (j), 定义:

$$\text{RawGIS}^{(j)} = C_{\text{raw}}^{(j)}.$$

数据集层面的 Raw GIS 为:

$$\text{RawGIS} \frac{1}{N} \sum_{j=1}^N \text{RawGIS}^{(j)}.$$

Raw GIS 高说明模型答案随图干预改变, 但这只是图忠实性的必要条件, 不是充分条件。若正确答案在两个反事实图中都位于显著位置, 模型可以仅凭位置启发式获得较高 Raw GIS。

Position-Controlled GIS

Position-Controlled GIS, 简称 PC-GIS, 要求模型在所有位置控制序列化下都能正确随图改变答案。对样本 (j), 定义:

$$\text{PCGIS}^{(j)} \prod_{\alpha \in \Sigma} C_\alpha^{(j)}.$$

也就是说, 只有当模型在 endpoint-first、endpoint-middle、endpoint-last 和 decoy-last 四种条件下都对反事实图对作出正确响应时, ($\text{PCGIS}^{\{j\}} = 1$)。

数据集层面的 PC-GIS 为:

$$\text{PCGIS} \frac{1}{N} \sum_{j=1}^N \text{PCGIS}^{(j)}.$$

PC-GIS 比 Raw GIS 更严格，因为它要求模型的图干预敏感性不能依赖某个特定的终点位置。

Graph-Following Inflation

Graph-Following Inflation, 简称 GFI, 用于量化 Raw GIS 中可能被位置线索虚高的部分。对样本 (j), 定义:

$$\text{GFI}^{(j)} = \text{RawGIS}^{(j)} - \text{PCGIS}^{(j)}.$$

数据集层面的 GFI 为:

$$\text{GFI} \frac{1}{N} \sum_{j=1}^N \text{GFI}^{(j)}.$$

当 Raw GIS 高而 PC-GIS 低时, GFI 会升高, 说明模型在默认序列化下看似能随图改变答案, 但这种能力无法通过位置控制检验。本文将这种现象称为图跟随虚胖: 模型的表面图敏感性被终点位置等文本捷径高估。

Endpoint Anchoring Rate

为了直接测量模型是否被显著位置吸引, GIF 还报告 decoy-last Endpoint Anchoring Rate, 简称 EAR.

设 ($d^{(j)}$) 是第 (j) 个样本在 decoy-last 序列化中被放在最后的诱饵节点。该节点满足:

$$d^{(j)} \neq y^{(j)}.$$

定义样本级 decoy-last EAR 为:

$$\text{EAR}_{\text{decoy}}^{(j)} \mathbb{1} \left[\hat{a}_{\text{decoy}}^{(j)} = d^{(j)} \right],$$

其中 ($\hat{a}_{\text{decoy}}^{(j)}$) 表示模型在 decoy-last 条件下的输出答案。数据集层面取平均:

$$\text{EAR}_{\text{decoy}} \frac{1}{N} \sum_{j=1}^N \text{EAR}_{\text{decoy}}^{(j)}.$$

decoy-last EAR 是比 endpoint-last 选择率更干净的锚定指标, 因为在 decoy-last 条件下, 最后出现节点被保证为错误诱饵。若模型仍输出该节点, 则更直接地说明它受到位置线索驱动。

3.4 Path-Level Metrics: TAC, PathValid, PathGoldExact, and FailureHop

答案层诊断仍然不充分。即使模型给出正确答案, 或者答案随图干预改变, 也不能说明它真的沿图走了一条合法路径。特别是在结构化 CoT 或 JSON path 输出中, 模型可能生成一条看似完整、且终点与答案一致的路径, 但其中某些边并不存在于输入图中。

因此, GIF 进一步引入路径层指标, 用于区分 “文本自治” 与 “图上合法”。

设模型输出路径为:

$$\hat{p} = (\hat{v}_0, \hat{v}_1, \dots, \hat{v}_k),$$

输出答案为：

$$\hat{a}.$$

Trace-Answer Consistency

Trace-Answer Consistency, 简称 TAC, 检查路径终点是否等于最终答案：

$$\text{TAC}(\hat{p}, \hat{a}) \mathbb{1} [\hat{a} = \hat{v}_k].$$

TAC 高说明模型输出内部自治：它声称的路径终点与最终答案一致。然而，TAC 不检查路径是否真的存在于图上。

Path Validity

PathValid 检查模型输出路径是否为输入图上的合法 (k)-hop walk：

$$\text{PathValid}(\hat{p}, G) \mathbb{1} [\hat{v}_0 = s; \wedge; |\hat{p}| = k + 1; \wedge; \forall i \in 0, \dots, k - 1, (\hat{v}_i, \hat{v}_{i+1}) \in E].$$

该指标同时检查三件事：路径是否从指定起点出发，路径长度是否正确，以及每一步是否沿图中真实存在的边移动。

Self-Consistent but Illegal Trace Gap

为了量化“轨迹自治但图上非法”的失败模式，定义非法路径差距：

$$\Delta_{\text{illegal}} = \text{TAC} - \text{PathValid}.$$

当 TAC 很高而 PathValid 很低时，

$$\Delta_{\text{illegal}}$$

会很大，说明模型能够生成与答案一致的结构化轨迹，却没有生成输入图上的合法路径。该指标是本文路径层诊断的核心信号。

这一点可写成：

$$\text{TAC} = 1; \nRightarrow; \text{PathValid} = 1.$$

即，轨迹与答案一致，并不意味着轨迹忠实于输入图结构。

Path Gold Exact

在唯一黄金路径设定下，进一步定义 PathGoldExact：

$$\text{PathGoldExact}(\hat{p}, p^*) \mathbb{1} [\hat{p} = p^*].$$

PathGoldExact 比 PathValid 更严格。PathValid 只要求路径是合法的 (k)-hop walk；PathGoldExact 要求模型路径完全等于生成器标注的黄金路径。在当前唯一黄金路径设置下，二者数值通常高度一致；但在多合法路径设置中，PathValid 和 PathGoldExact 可以分离。

若未来扩展到多条合法黄金路径，可定义：

$$\text{PathGoldExact}(\hat{p}, P^*) \mathbb{1} [\hat{p} \in P^*].$$

FailureHop

为了定位模型从哪一步开始偏离图结构，定义 FailureHop。若模型输出路径长度不足、起点错误或格式错误，则将其记录为相应错误类型；若路径长度足够且起点正确，则 FailureHop 为第一条非法边所在的 hop：

$$\text{FailureHop}(\hat{p}, G) = \min \{i \in \{1, \dots, k\} : (\hat{v}_{i-1}, \hat{v}_i) \notin E\}.$$

若全部边均合法，则 FailureHop 记为 pass。

FailureHop 的作用是把“路径错了”细化为“从第几跳开始错”。这对于分析分叉图尤其重要，因为模型可能反复在早期分叉点、终端转移或特定状态更新位置失败。后续实验将使用 FailureHop 区分局部结构瓶颈与整体状态追踪失败。

3.5 Symbolic Verifier and Retry

GIF 的核心是诊断模型原始行为，但我们还引入符号 verifier-retry，用于研究外部结构反馈是否能修复非法路径，以及这种修复是否受底座模型能力限制。

给定第 (j) 个样本在第 (t) 次尝试中的输出：

$$(\hat{p}_{j,t}, \hat{a}_{j,t}),$$

符号验证器检查以下条件：

1. 输出是否可解析为指定格式；
2. 路径是否从起点 (s) 出发；
3. 路径长度是否为 (k+1)；
4. 每一条边是否属于输入图 (E)；
5. 路径终点是否等于模型答案 $\hat{a}_{j,t}$ 。

需要强调的是，在线 verifier feedback 不泄露黄金答案 (y)，也不直接告诉模型正确下一跳。黄金答案正确性只在离线评估阶段计算。

若验证失败，验证器返回结构性错误反馈，例如：

Your path is invalid at hop 3:
edge (J9E9, W6S2) does not exist in the graph.
Please retry with a valid k-hop path from the given start node.

模型随后重新生成路径和答案，最多尝试 (K) 次。记第 (j) 个样本第一次通过验证的尝试编号为：

$$\tau_j \min t : \text{VerifierPass}_{j,t} = 1.$$

若在 (K) 次内始终未通过，则记为：

$$\tau_j = \infty.$$

定义 pass@K :

$$\text{pass@K} \frac{1}{N} \sum_{j=1}^N \mathbb{1}[\tau_j \leq K].$$

除了 pass@K , GIF 还记录每次尝试的延迟、错误类型和 FailureHop。设第 (j) 个样本第 (t) 次尝试的延迟为 $\ell_{j,t}$, 则预算 (K) 下的累计延迟为:

$$L_j^{(K)} = \sum_{t=1}^{\min(\tau_j, K)} \ell_{j,t},$$

其中若 $\tau_j = \infty$, 则累计全部 (K) 次尝试。数据集平均累计延迟为:

$$L^{(K)} = \frac{1}{N} \sum_{j=1}^N L_j^{(K)}.$$

因此, verifier-retry 不仅报告最终修复率, 还报告成本—效果曲线:

$$(\text{pass@1}, L^{(1)}), (\text{pass@2}, L^{(2)}), \dots, (\text{pass@K}, L^{(K)}).$$

对于在 (K) 次后仍失败的样本, 我们进一步统计其最终错误类型和最终 FailureHop 分布:

$$e_{j,K} : \tau_j = \infty,$$

$$h_{j,K} : \tau_j = \infty.$$

这一步用于判断剩余失败是随机噪声, 还是稳定结构瓶颈。如果模型在多次 retry 后仍反复卡在同一 hop, 则说明 verifier 能发现错误, 但模型本身缺少修复该错误所需的状态更新能力。

因此, verifier-retry 在 GIF 中有双重作用。它既是一个低成本修复机制, 用于检验外部符号反馈能否提高路径合法性; 也是一个诊断工具, 用于区分“模型能在反馈下修正路径”与“模型反复卡在单一结构转移处”这两种情况

4 Experimental Setup

本节说明实验如何运行, 包括数据集构造、模型与 prompt 设置、输出解析、统计方法和延迟记录。GIF 的指标定义已在第 3 节给出, 本节不再重复框架定义, 只描述具体实验配置。

4.1 Synthetic Graph Construction

我们构造合成符号图任务, 而不是直接使用自然语言知识图谱。每个节点使用随机符号标识, 例如 J9E9、W6S2 等, 以削弱实体名称、常识关联和参数记忆带来的语义先验。这样, 模型若要完成任务, 主要必须依赖输入中的显式图结构。

每个基础样本包含一个起点 (s)、一个跳数 (k)、一条唯一黄金路径 (p^*) 和一个黄金终点 (y)。模型任务是从 (s) 出发, 沿图中合法有向边恰好走 (k) 跳, 并输出最终节点。

对于每个基础样本, 我们构造一组反事实图对:

$$(G_1, G_2),$$

其中两张图共享相同起点 (s) 与相同跳数 (k)，但拥有不同黄金终点：

$$y_1 \neq y_2.$$

每张图进一步生成四种位置控制序列化：

$$\sigma_{\text{first}}, \quad \sigma_{\text{middle}}, \quad \sigma_{\text{last}}, \quad \sigma_{\text{decoy}}.$$

因此，每个基础样本对应：

$$2 \times 4 = 8$$

个模型查询。每个数据集包含 (N=200) 个基础样本，因此每个 dataset 在单一模型和单一 prompt 下包含：

$$200 \times 2 \times 4 = 1600$$

个查询。

本文使用六个主数据集：

- **chain_3hop_N200**: Topology: Chain; Hop length: 3; Base samples: 200; Queries per model/prompt: 1600
- **chain_4hop_N200**: Topology: Chain; Hop length: 4; Base samples: 200; Queries per model/prompt: 1600
- **chain_5hop_N200**: Topology: Chain; Hop length: 5; Base samples: 200; Queries per model/prompt: 1600
- **chain_6hop_N200**: Topology: Chain; Hop length: 6; Base samples: 200; Queries per model/prompt: 1600
- **branching_4hop_N200**: Topology: Branching; Hop length: 4; Base samples: 200; Queries per model/prompt: 1600
- **branching_12hop_N200**: Topology: Branching; Hop length: 12; Base samples: 200; Queries per model/prompt: 1600

Chain graphs

Chain graphs 用于检测 answer-only 接口下的 endpoint-position shortcut。在链式图中，从起点到答案存在唯一黄金路径，结构相对简单，因此如果模型表现出较高 Raw GIS 但 PC-GIS 为 0，就能较清楚地说明模型依赖了终点位置，而不是真正稳定地使用图结构。

我们使用 chain-3hop 到 chain-6hop 进行长度扫描。该设置用于观察弱提示下的答案层失败是否随 hop 长度变化，并比较不同模型是否表现出稳定位置捷径、部分捷径依赖或图干预敏感性崩溃。

Branching graphs

Branching graphs 用于检测路径层状态追踪能力。与链式图不同，分叉图在中间节点处包含多个候选分支，模型不能只依赖最后出现节点或局部显著节点，而必须持续维护当前节点状态，并在每一步选择图中真实存在的后继边。

我们使用 branching-4hop 和 branching-12hop 两档难度。branching-4hop 作为中间难度设置，用于观察从简单链式读取到分叉状态追踪之间的过渡；branching-12hop 作为主要困难设置，用于放大长程状态追踪失败和非法路径生成。

Graph validation

所有图在进入实验前均通过符号程序验证。验证内容包括：

1. (G_1) 与 (G_2) 均包含完整图结构；
2. 两张图共享相同起点 (s) 与跳数 (k)；
3. 两张图的黄金终点不同，即 $y_1 \neq y_2$ ；
4. 每张图中从 (s) 出发恰好 (k) 跳的黄金终点唯一；
5. 当前实验设定下黄金路径唯一；
6. decoy 节点不是合法答案；
7. 四种位置控制序列化满足 endpoint-first、endpoint-middle、endpoint-last 与 decoy-last 的位置约束；
8. 边列表中不存在破坏唯一性的额外路径。

只有通过全部检查的样本才进入最终评测。

4.2 Models and Prompts

Models

本文评估四个主要模型配置：

- **DeepSeek-V4-Flash**: Mode: no-thinking; Role: 低延迟主模型，用于观察位置捷径、非法路径和 verifier-retry 的修复上限
- **DeepSeek-V4-Pro**: Mode: no-thinking; Role: 更强模型的有限推理预算基线
- **DeepSeek-V4-Pro**: Mode: thinking; Role: 高推理预算对照，用于检验内部 reasoning budget 是否能解决路径非法问题
- **Qwen Max**: Mode: no-thinking; Role: 跨模型家族对照，用于检验失败模式是否稳定存在
- **GPT-5.4-mini**: Mode: no-thinking; Role: 额外闭源模型对照，用于观察位置捷径和路径非法是否跨模型出现

DeepSeek-V4-Pro thinking 只在关键困难设置上运行，即：

branching-12hop + jsoncot_strict

该设置用于回答：额外内部推理预算是否能显著提高路径合法性，以及这种提升需要多高延迟成本。

verifier-retry 也只在困难设置上运行：

branching-12hop + jsoncot_strict, $K = 5$.

该设置用于回答：外部符号验证能否在低于 thinking mode 的延迟下部分修复非法路径，以及其收益是否依赖底座模型本身的图跟随能力。

Prompt interfaces

本文使用五类 prompt，其中前三类用于主任务，后两类用于 prior control。

- **direct_minimal**: Output: answer only; Purpose: 检测 answer-only 接口下的 endpoint-position shortcut
- **jsoncot_basic**: Output: JSON path + answer; Purpose: 观察显式路径输出是否降低位置锚定，并开启路径级诊断
- **jsoncot_strict**: Output: strict JSON path + answer; Purpose: 强制路径长度、起点、逐边合法性和 answer-path consistency 的结构化输出格式

- **prior_only**: Output: answer only, no graph; Purpose: 检查无图条件下的固定答案先验
- **candidate_only_prior**: Output: answer only, candidates but no graph; Purpose: 检查候选集合偏好是否能解释答案命中

`direct_minimal` 只要求模型输出最终答案，不要求给出路径。因此，该 prompt 主要用于答案层指标，包括 Accuracy、Raw GIS、PC-GIS、GFI 和 decoy-last EAR。

`jsoncot_basic` 要求模型输出 JSON 格式的路径与最终答案，例如：

```
{
  "path": ["S0", "...", "Y"],
  "answer": "Y"
}
```

该设置让路径级指标可被计算，包括 TAC、PathValid、PathGoldExact 和 FailureHop。

`jsoncot_strict` 在 `jsoncot_basic` 基础上加入更严格的格式约束。模型必须输出严格 JSON，不得包含额外文本；路径必须包含 $(k+1)$ 个节点，必须从起点 (s) 开始，答案必须等于路径最后一个节点。需要注意的是，这些约束只规定输出格式，并不保证模型实际生成的每条边都属于输入图。因此，所有结构化输出仍需进行符号路径验证。

`prior_only` 和 `candidate_only_prior` 不提供完整图结构，用于排除答案先验和候选偏好。若模型在无图或仅候选条件下已经能稳定命中正确答案，则相应样本可能被视为 prior-confounded，并从主分析中剔除或单独报告。

Decoding configuration

所有主实验使用确定性解码：

$$\text{temperature} = 0.$$

若某些模型 API 不暴露 temperature 控制，则使用 provider 默认确定性设置，并在实验记录中标注。所有请求记录模型名称、prompt mode、dataset、serialization variant、latency、parse status、timeout status 和原始输出路径。

4.3 Evaluation and Statistics

Output parsing

对于 answer-only prompt，我们解析模型输出中的最终答案 $\hat{a}$ 。若模型输出多个候选或无法定位唯一答案，则记为 parse failure。

对于 structured prompt，我们解析：

$$(\hat{p}, \hat{a}).$$

其中 $\hat{p}$ 为模型输出路径， $\hat{a}$ 为模型最终答案。若 JSON 无法解析、缺少必要字段、路径不是 list、节点格式不合法，或输出包含无法恢复的额外文本，则记为 format failure。format failure 在路径级指标中视为无效路径，并在错误类型统计中单独记录。

Answer-level evaluation

答案层评估使用第 3 节定义的指标：

- Accuracy;
- Raw GIS;
- PC-GIS;

- GFI;
- decoy-last EAR。

Raw GIS 和 PC-GIS 在同一批反事实图对上逐样本配对计算。GFI 在样本级先计算：

$$\text{GFI}^{(j)} = \text{RawGIS}^{(j)} - \text{PCGIS}^{(j)},$$

再在数据集层面取平均。

Path-level evaluation

路径层评估使用第 3 节定义的指标：

- TAC;
- PathValid;
- PathGoldExact;
- Δ_{illegal} ;
- FailureHop;
- error type distribution。

TAC 检查答案是否等于路径末节点；PathValid 检查路径是否从起点出发、长度是否为 (k+1)、每条边是否存在于输入图中；PathGoldExact 检查路径是否完全等于唯一黄金路径；FailureHop 记录第一条非法边所在的 hop。

对于路径过短、路径过长、起点错误、格式错误和 trace-answer mismatch，分别记录对应错误类型。对于路径长度足够且起点正确但某一步边不存在的输出，记录为：

illegal_edge_at_hop_i

其中 (i) 表示第一条非法边所在 hop。

Verifier-retry evaluation

verifier-retry 实验在 branching-12hop + jsoncot_strict 上运行，最大重试次数为：

$$K = 5.$$

每次尝试后，符号验证器检查输出格式、路径起点、路径长度、逐边合法性以及 answer-path consistency。若验证失败，验证器只返回结构性错误反馈，例如非法边所在 hop 或路径长度错误，不泄露 gold answer 或正确下一跳。

对每个样本记录：

- 每次尝试的输出路径与答案；
- 每次尝试的 latency；
- 每次尝试的 verifier pass/fail；
- 每次失败的 FailureHop；
- 每次失败的 error type；
- 第一次通过验证的尝试编号 τ_j 。

报告 pass@K:

$$\text{pass@K} \frac{1}{N} \sum_{j=1}^N \mathbb{1}[\tau_j \leq K].$$

同时报告每个 (K) 档的累计平均延迟:

$$L^{(K)} \frac{1}{N} \sum_{j=1}^N \sum_{t=1}^{\min(\tau_j, K)} \ell_{j,t},$$

其中 $\ell_{j,t}$ 是第 (j) 个样本第 (t) 次尝试的延迟。若样本在 (K) 次内未通过验证, 则累计全部 (K) 次尝试。

对于 (K=5) 后仍失败的样本, 我们统计最终 FailureHop 分布、最终 error type 分布, 以及 repeated_same_hop 比例。repeated_same_hop 表示模型在多次 retry 中反复失败于同一 hop, 用于判断剩余失败是否为稳定结构瓶颈。

verifier-retry 结果对 DeepSeek-V4-Flash 报告 3 次独立运行的平均值; 对 Qwen Max 报告 2 次独立运行的平均值。误差棒或置信区间基于这些重复运行和样本级统计计算。

Confidence intervals

对主实验中的样本级指标报告 bootstrap 置信区间。由于模型调用使用确定性解码, bootstrap 反映的是样本选择带来的不确定性, 而不是 decoding randomness。

对 Raw GIS、PC-GIS、GFI 等配对指标, 使用 paired bootstrap: 以基础反事实图对为单位重采样, 并在每个重采样集合内重新计算指标。对 Accuracy、TAC、PathValid、PathGoldExact、 Δ_{illegal} 等样本级比例指标, 使用 sample-level bootstrap。

本文默认使用 95% confidence intervals。对于 verifier-retry 的 pass@K 和 latency 曲线, 报告多 run 均值, 并在图中显示运行间误差或样本级不确定性。

Latency and timeout handling

所有模型调用记录 wall-clock latency。对于普通推理, latency 记录单次请求耗时; 对于 verifier-retry, latency 记录每次尝试的耗时, 并累计至通过验证或达到最大重试次数 (K)。

若请求超过预设 timeout, 则记为 timed_out。timeout 输出不计为正确答案, 也不计为合法路径; 在路径级指标中视为无效输出, 并在错误统计中单独保留。latency 分析报告平均值、中位数和高分位数, 以避免少数极端慢请求掩盖整体趋势。

Reproducibility

所有实验保存以下信息:

- dataset name;
- sample id;
- graph id;
- model name;
- prompt mode;
- serialization variant;
- gold answer;

- gold path;
- model output;
- parsed answer;
- parsed path;
- parse status;
- metric values;
- latency;
- timeout status;
- verifier feedback and retry history, when applicable.

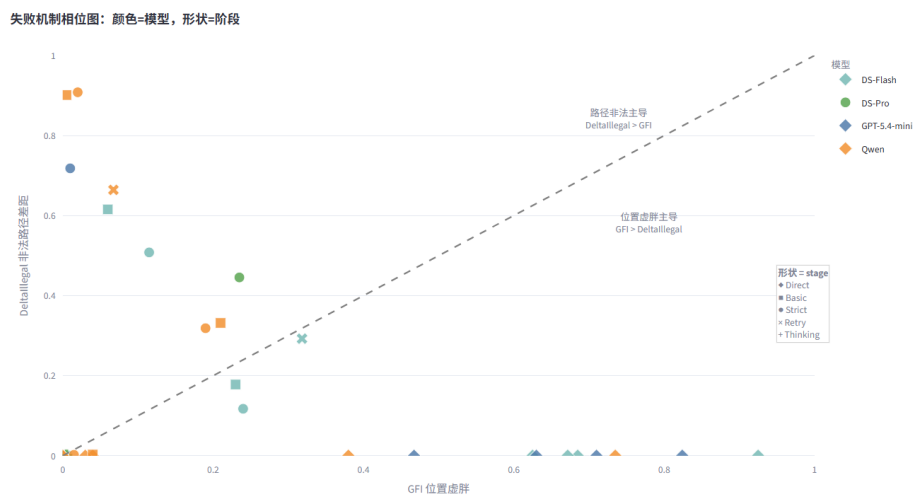
图生成器、序列化脚本、输出解析器和符号 verifier 均使用同一套节点与边表示，以避免评估阶段出现格式不一致。所有主结果均基于通过图结构校验的数据集计算。

5 Results

本节报告 GIF 在符号图任务上的主要结果。整体发现可以概括为三点。第一，prior controls 表明模型不能仅凭答案先验或候选偏好完成任务。第二，在 chain graphs 的弱提示条件下，Raw GIS 会被 endpoint-position shortcut 显著虚高；位置控制后，这种表面图跟随能力会崩塌。第三，在 branching graphs 的结构化输出条件下，位置捷径被削弱，但更深层的路径非法问题暴露出来：模型经常生成与答案自治、但图上非法的路径。

除非特别说明，本节中的比例指标均以百分比报告。

C. 失败机制相位图：每个设置对应哪个区域



5.1 Prior controls rule out answer priors

首先检查模型是否能够在无图或仅给候选节点的情况下猜中答案。若模型在 prior-only 或 candidate-only prior 条件下已经能稳定输出正确答案，那么主任务上的成功可能来自答案先验、候选偏好或格式偏好，而不是图结构追踪。

在 branching-12hop 上，prior controls 基本排除了这一解释。prior-only 条件下，DeepSeek-V4-Flash 和 Qwen Max 的准确率均为 0%。candidate-only prior 条件下，DeepSeek-V4-Flash 的准确率为 0.25%，Qwen Max 为 0%。这一结果低于或接近随机候选基线：

$$\frac{1}{84} \approx 1.19$$

因此，后续主实验中的成功或失败不能由无图答案先验解释。模型必须利用输入图结构，才可能稳定命中正确终点。

Finding 1. Prior controls rule out answer priors: without graph structure, models almost never identify the correct endpoint on branching-12hop.

5.2 Weak prompts induce answer-level positional shortcuts

接下来分析 answer-only 弱提示是否会诱发答案层位置捷径。我们使用 chain graphs 作为浅层诊断环境，因为链式结构中从起点到终点只有唯一简单路径，若正确终点总被放在显著位置，模型可能不真正沿图推理，而是直接选择最后或最显著的节点。

Chain GFI 非法路径差距 pass@K 曲线 失败 hop Raw / PC / GFI

Chain direct_minimal: GFI 位置虚胖

Chain tasks under direct_minimal: GFI position inflation

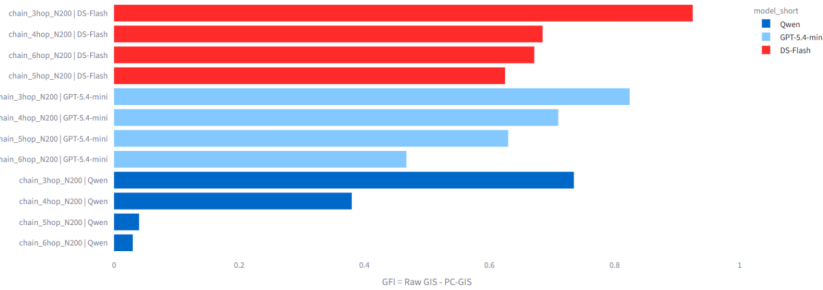
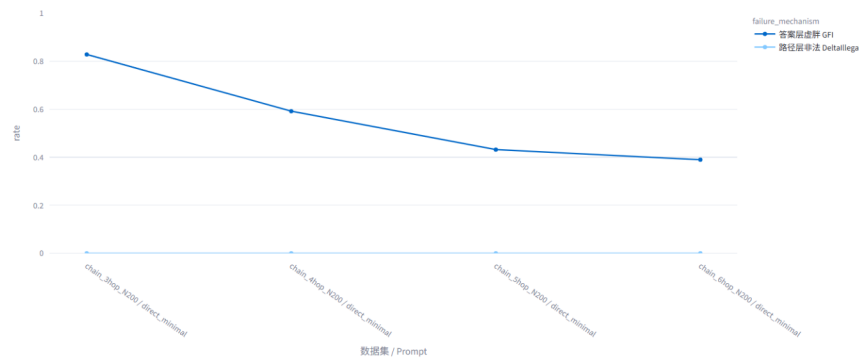


Table 1 显示 chain-4hop + direct_minimal 下的结果。DeepSeek-V4-Flash 的 Raw GIS 达到 68.5%，但 PC-GIS 为 0%，因此 GFI 也达到 68.5%。这说明模型在默认 endpoint-last 条件下看似能随图改变答案，但这种能力完全无法通过位置控制检验。decoy-last EAR 进一步达到 63.8%，说明当错误诱饵被放在最后时，模型经常被最后位置吸引。

B3. 按数据集 × Prompt 汇总：GFI vs 非法路径差距

按数据集 × Prompt 汇总：答案层虚胖 vs 路径层非法



Qwen Max 也呈现相同方向：Raw GIS 为 38.0%，PC-GIS 为 0%，GFI 为 38.0%。不过 Qwen 的 decoy-last EAR 只有 19.7%，说明它并不是单纯稳定地选择最后节点；在更长链上，它更接近图干预敏感性崩溃，即模型对图变化本身逐渐不敏感。

- **DeepSeek-V4-Flash:** Accuracy: 29.5; Raw GIS: 68.5 [62.0, 75.0]; PC-GIS: 0.0 [0.0, 0.0]; GFI: 68.5 [62.0, 75.0]; decoy-last EAR: 63.8 [59.2, 68.2]
- **Qwen Max:** Accuracy: 46.1; Raw GIS: 38.0 [31.5, 45.0]; PC-GIS: 0.0 [0.0, 0.0]; GFI: 38.0 [31.5, 45.0]; decoy-last EAR: 19.7 [15.8, 23.8]

Table 1. Chain-4hop under direct_minimal. Raw GIS can be substantially inflated by endpoint-position shortcuts; after position control, PC-GIS collapses to zero.

进一步的 chain 长度扫描显示, 不同模型呈现出递进式答案层失败。DeepSeek-V4-Flash 在 chain-3hop 到 chain-6hop 上都保持较高 GFI 与 decoy-last EAR, 说明其弱提示行为稳定受到 endpoint-position shortcut 影响。GPT-5.4-mini 表现为中间型: 它也受到位置线索影响, 但随 hop 增长逐步下降。Qwen Max 则在较长链上 Raw GIS 急剧降低, 说明其主要问题逐渐从位置捷径转向图干预敏感性崩溃。

具体而言, Raw GIS 在 chain-3hop 到 chain-6hop 上呈现如下趋势:

- **3:** DeepSeek-V4-Flash: 92.5; GPT-5.4-mini: 86.4; Qwen Max: 85.5
- **4:** DeepSeek-V4-Flash: 68.5; GPT-5.4-mini: 71.0; Qwen Max: 38.0
- **5:** DeepSeek-V4-Flash: 62.5; GPT-5.4-mini: 63.0; Qwen Max: 4.0
- **6:** DeepSeek-V4-Flash: 67.2; GPT-5.4-mini: 46.7; Qwen Max: 3.0

对应的 decoy-last EAR 也显示出类似差异:

- **3:** DeepSeek-V4-Flash: 81.3; GPT-5.4-mini: 44.3; Qwen Max: 24.0
- **4:** DeepSeek-V4-Flash: 63.8; GPT-5.4-mini: 58.3; Qwen Max: 19.8
- **5:** DeepSeek-V4-Flash: 75.8; GPT-5.4-mini: 66.3; Qwen Max: 9.0
- **6:** DeepSeek-V4-Flash: 77.5; GPT-5.4-mini: 62.0; Qwen Max: 10.5

这些结果说明, 弱提示并不只产生一种失败。DeepSeek-V4-Flash 的失败更像稳定的位置捷径; GPT-5.4-mini 处于中间状态; Qwen Max 在长链上则表现为 sensitivity collapse。仅看 Raw GIS 会把这些不同机制混在一起, 而 GFI 与 EAR 能进一步区分它们。

Finding 2. Weak answer-only prompts induce answer-level false faithfulness: Raw GIS may be high under endpoint-last serialization, but PC-GIS collapses after position control.

5.3 Structured output removes chain shortcuts but exposes illegal traces

结构化输出显著改变了 chain graphs 上的行为。在 chain-4hop + jsoncot_strict 下, DeepSeek-V4-Flash 几乎完全解决任务, Accuracy、Raw GIS、PC-GIS 和 PathGoldExact 均为 100%。Qwen Max 也达到 99.8% Accuracy、100.0% Raw GIS、98.5% PC-GIS 和 99.8% PathGoldExact。与 direct_minimal 相比, 严格结构化输出几乎消除了 chain 上的答案层位置捷径。

- **DeepSeek-V4-Flash:** Accuracy: 100.0; Raw GIS: 100.0; PC-GIS: 100.0; GFI: 0.0; PathGoldExact: 100.0
- **Qwen Max:** Accuracy: 99.8; Raw GIS: 100.0; PC-GIS: 98.5; GFI: 1.5; PathGoldExact: 99.8

Table 2. Chain-4hop under jsoncot_strict. Structured output removes answer-level shortcuts on simple chain graphs.

然而, 这并不意味着结构化输出保证图忠实性。当任务转入 branching-12hop 后, jsoncot_strict 暴露出更深层的路径非法问题。Table 3 显示, 多个模型都能生成与答案高度自治的结构化路径, 但路径本身并不合法。

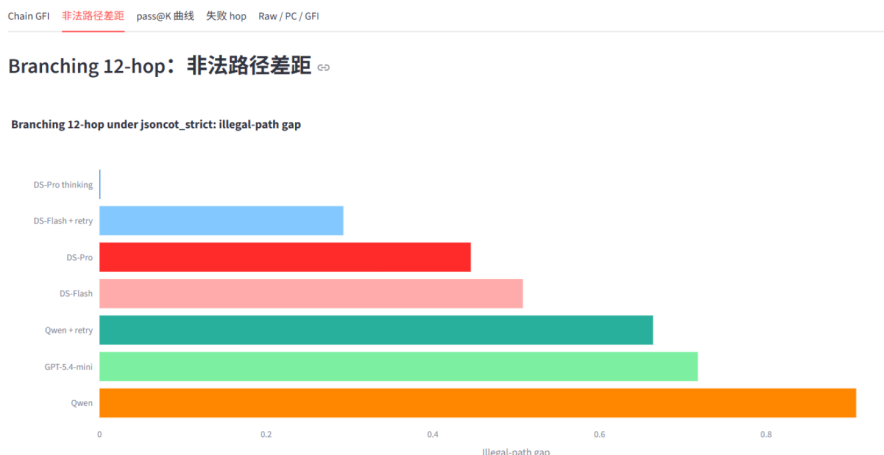
DeepSeek-V4-Flash 的 TAC 达到 99.3%, 说明其答案几乎总是等于路径末节点; 但 PathValid 和 PathGoldExact 只有 48.5%, 导致非法路径差距 Δ_{illegal} 达到 50.8%。Qwen Max 的情况更极端: TAC 为 99.2%, 但 PathGoldExact 只有 8.4%, Δ_{illegal} 高达 90.8%。这说明 Qwen Max 非常擅长保持输出内部自治, 却几乎无法保证路径逐边合法。

GPT-5.4-mini 的 PathGoldExact 也很低, 为 8.1%, 但其 TAC 只有 79.9%。这表明 GPT-5.4-mini 与 Qwen Max 的失败表面上同样表现为低路径正确率, 但机制不同: Qwen Max 是“高度自治但非法”, GPT-5.4-mini 则包含更多基础输出不自治或轨迹构造失败。

- **DeepSeek-V4-Flash:** Accuracy: 50.2; PC-GIS: 0.0; TAC: 99.3 [98.9, 99.7]; PathValid: 48.5 [46.1, 50.9]; PathGoldExact: 48.5 [46.1, 50.9]; (Δ_{illegal}): 50.8 [48.5, 53.2]

- **DeepSeek-V4-Pro no-thinking:** Accuracy: 56.3; PC-GIS: 2.0; TAC: 99.9 [99.8, 100.0]; PathValid: 55.3 [52.5, 58.1]; PathGoldExact: 55.3 [52.5, 58.1]; (Δ_{illegal}): 44.6 [41.8, 47.4]
- **Qwen Max:** Accuracy: 13.7; PC-GIS: 0.0; TAC: 99.2 [98.7, 99.6]; PathValid: 8.4 [6.9, 9.9]; PathGoldExact: 8.4 [6.9, 9.9]; (Δ_{illegal}): 90.8 [89.2, 92.3]
- **GPT-5.4-mini:** Accuracy: 8.3; PC-GIS: 0.0; TAC: 79.9 [77.9, 81.8]; PathValid: 8.1 [6.6, 9.6]; PathGoldExact: 8.1 [6.6, 9.6]; (Δ_{illegal}): 71.8 [69.4, 74.1]

Table 3. Branching-12hop under jsoncot_strict. Models often produce self-consistent traces, but these traces are not valid paths in the input graph.



Branching-4hop 作为中间难度设置进一步支持这一解释。与 chain-4hop 不同, branching-4hop 已经引入分叉结构, 因此模型必须维护当前节点状态, 而不能只读取线性链条。结果显示, chain-4hop + jsoncot_strict 几乎被完全解决; branching-4hop 开始出现非零非法路径差距; 到 branching-12hop 时, 该差距显著放大。这说明困难不只是来自 JSON 输出格式, 也不只是来自 hop 数增加, 而是来自模型在分叉结构中持续维护状态的能力不足。

Finding 3. Structured output removes simple chain shortcuts, but exposes trace-level false faithfulness on branching graphs: high TAC does not imply high PathValid.

5.4 Thinking reduces failures at high latency

进一步比较 DeepSeek-V4-Pro 的 no-thinking 与 thinking 模式。结果显示, thinking 几乎解决 branching-12hop + jsoncot_strict, 但代价是显著更高延迟。

在 no-thinking 模式下, DeepSeek-V4-Pro 的 Accuracy 为 56.3%, PathGoldExact 为 55.3%, PC-GIS 仅为 2.0%, 仍存在明显路径非法和图跟随不稳定问题。开启 thinking 后, Accuracy 提升到 99.9%, Raw GIS 为 100.0%, PC-GIS 为 99.5%, GFI 降到 0.5%, PathGoldExact 达到 99.9%。这说明在足够高的内部推理预算下, 模型能够几乎完全消除路径非法问题。

然而, thinking 的延迟成本也显著增加。DeepSeek-V4-Pro no-thinking 平均延迟约为 2.4 秒, 而 thinking 平均延迟为 46.7 秒, 中位数为 42.4 秒, p95 达到 203 秒。平均延迟约增加 19 倍。

- **DeepSeek-V4-Pro no-thinking:** Accuracy: 56.3; Raw GIS: 25.5; PC-GIS: 2.0; GFI: 23.5; PathGoldExact: 55.3; Avg latency: 2.4s
- **DeepSeek-V4-Pro thinking:** Accuracy: 99.9; Raw GIS: 100.0; PC-GIS: 99.5; GFI: 0.5; PathGoldExact: 99.9; Avg latency: 46.7s

Table 4. Thinking mode on branching-12hop + jsoncot_strict. Extra reasoning budget nearly solves the task, but at much higher latency.

这一结果有两个含义。第一, branching-12hop 任务本身并非不可解; 强模型在高推理预算下可以接近完美完成。第二, no-thinking 模式下的失败不是简单格式问题, 而是有限推理预算下的状态追踪失败。thinking 可以缓解该问题, 但其延迟成

本较高。

Finding 4. Thinking reduces both answer-level and path-level failures, but incurs a large latency cost.

5.5 Verifier-retry partially repairs illegal paths

最后分析外部符号 verifier-retry 是否能以较低成本修复非法路径。该实验在 branching-12hop + jsoncot_strict 上运行，最大重试次数为 (K=5)。验证器只返回结构性错误反馈，例如路径长度错误或某一跳边不存在；它不泄露 gold answer，也不告诉模型正确下一跳。

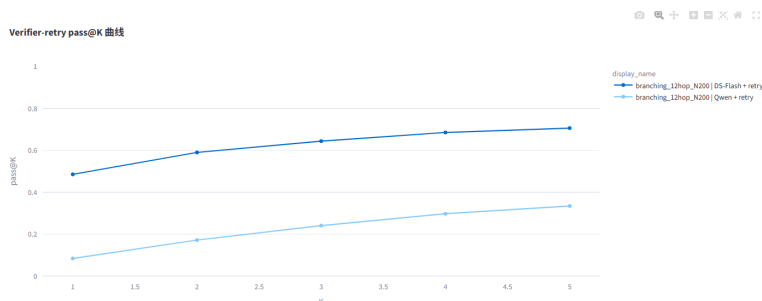


Table 5 显示 pass@K 曲线。DeepSeek-V4-Flash 从 pass@1 的 48.6% 提升到 pass@5 的 70.6%，累计延迟为 4.42 秒。Qwen Max 从 pass@1 的 8.4% 提升到 pass@5 的 33.4%，累计延迟为 12.12 秒。

- **DeepSeek-V4-Flash + retry:** pass@1: 48.6; pass@2: 59.0; pass@3: 64.4; pass@4: 68.6; pass@5: 70.6; Latency@5: 4.42s
- **Qwen Max + retry:** pass@1: 8.4; pass@2: 17.2; pass@3: 24.1; pass@4: 29.8; pass@5: 33.4; Latency@5: 12.12s

Table 5. Verifier-retry on branching-12hop + jsoncot_strict. pass@K improves with retry budget, but gains saturate and depend strongly on the base model.

Verifier-retry 的结果表明，外部验证确实有用，但它并不能凭空创造图推理能力。DeepSeek-V4-Flash 的 pass@1 已接近 50%，说明模型本身已经具备一定图跟随能力，因此结构性反馈可以修复一部分错误。Qwen Max 的 pass@1 只有 8.4%，即使经过 5 次重试也只达到 33.4%，说明当底座模型状态追踪能力较弱时，反馈收益存在明显上限。

与 DeepSeek-V4-Pro thinking 相比，DeepSeek-V4-Flash verifier@5 以 4.42 秒达到 70.6% 的路径通过率，而 Pro thinking 以 46.7 秒达到 99.9% 的 PathGoldExact。也就是说，verifier-retry 不能替代 thinking，但能以不到十分之一的平均延迟恢复大量路径合法性。

pass@K 曲线也显示出边际收益递减。DeepSeek-V4-Flash 从 K=1 到 K=2 提升 10.4 个百分点，但从 K=4 到 K=5 只提升 2.0 个百分点。Qwen Max 也呈现类似饱和趋势。这说明 verifier 能发现错误，但后续修复仍依赖模型自身维护图状态和选择合法后继的能力。

Finding 5. Verifier-retry partially repairs illegal paths at lower latency, but its ceiling is determined by the base model's graph-following ability.

5.6 Summary of Results

本节结果支持 GIF 的核心论点：最终答案、Raw GIS 和结构化轨迹自治性都不足以证明模型忠实使用图结构。

在 chain graphs 的弱提示设置下，模型可能表现出较高 Raw GIS，但位置控制后 PC-GIS 崩塌，说明答案层图跟随能力被 endpoint-position shortcut 虚高。在 branching graphs 的结构化输出设置下，位置捷径被削弱，但模型仍会生成大量自治却非法的路径，说明 TAC 并不蕴含 PathValid。Thinking 可以几乎解决困难分叉图任务，但延迟成本高；verifier-retry 能以较低延迟部分修复非法路径，但其收益受底座模型能力限制。

整体而言，图推理评测需要同时检查三件事：答案是否在位置控制后仍随图干预改变，路径是否逐边合法，以及失败是否集中在特定结构位置。GIF 正是为同时诊断这三类问题而设计

6 Analysis

第 5 节已经报告了主要实验结果：弱提示下 chain graphs 会暴露答案层位置虚胖，结构化输出能缓解简单链式图上的 shortcut，但在 branching graphs 上又暴露出路径层非法轨迹；thinking 能显著缓解失败但延迟较高，verifier-retry 能部分修复非法路径但存在上限。

本节不再重复结果表，而是进一步解释这些现象背后的机制。我们关注三个问题：

1. 为什么 self-consistent traces 仍然可能是 illegal traces?
2. 为什么 verifier-retry 有效但会饱和?
3. FailureHop 和 repeated same-hop failure 如何揭示结构性瓶颈?

总体而言，分析表明，模型的关键失败并不是不会输出路径，也不是简单格式错误，而是无法稳定维护图上的当前状态，并在每一步选择真实存在的后继节点。

6.1 Why self-consistent traces can still be illegal

在 branching-12hop + jsoncot_strict 设置下，多数模型可以生成格式正确、答案与路径末节点一致的结构化输出。换言之，它们通常能满足：

$$\hat{a} = \hat{v}_k.$$

这解释了为什么 TAC 可以很高。模型知道 JSON 中应该包含 path 和 answer，也知道 answer 应该等于 path[-1]。但是，这种输出层面的自洽性并不等于路径在图上合法。真正的路径合法性要求模型在每一步都执行状态更新：

$$\hat{v}_{i+1} \in N^+(\hat{v}_i),$$

其中 $(N^+(\hat{v}_i))$ 表示当前节点 $\hat{v}_i$ 的出邻居集合。模型必须维护“当前节点是谁”，再从当前节点的合法后继中选择下一跳。只要某一步生成了图中不存在的边：

$$(\hat{v}_i, \hat{v}_{i+1}) \notin E,$$

整条路径就不再是输入图上的合法 walk。

因此，strict JSON-CoT 主要提升的是格式服从能力和局部文本自洽能力，而不是自动保证图状态追踪能力。模型可以生成一条看似完整、终点与答案一致的路径，但这条路径可能只是围绕答案构造出来的结构化解释，而不是模型真正沿图边移动得到的结果。

这与自然语言 CoT 中的 post-hoc rationalization 类似。在普通 CoT 中，模型可能先形成答案，再生成一段看起来支持该答案的解释；在图路径任务中，模型也可能先锁定某个答案节点，再补出一条看似通向该节点的路径。区别在于，这里的解释被包装成了结构化路径，因此更容易被误认为是可验证的推理轨迹。

GIF 的路径级诊断正是为了防止这种误判。TAC 只能说明模型输出内部不矛盾，不能说明路径中的每条边都存在。对于路径类图任务，真正关键的是逐边验证：

$$\forall i, ; (\hat{v}_i, \hat{v}_{i+1}) \in E.$$

因此，PathValid、PathGoldExact 和 FailureHop 不是附加指标，而是区分“结构化解释”和“图上合法路径”的必要条件。

6.2 Why verifier-retry saturates

Verifier-retry 能提高 pass@K, 说明外部符号反馈确实有用。但第 5 节也显示, retry 的收益会逐渐饱和, 而且不同模型的上限差异很大。这说明 verifier-retry 实际上区分了两种能力:

- **Error detection:** Provided by: Symbolic verifier; Meaning: 判断路径哪一步不合法
- **Error repair:** Provided by: Language model; Meaning: 根据输入图重新生成合法路径

验证器只能提供第一种能力。它可以告诉模型某一步边不存在、路径长度错误、起点错误或答案与路径末节点不一致。例如:

```
Your path is invalid at hop 3:  
edge (J9E9, W6S2) does not exist in the graph.
```

但是, 验证器不会泄露 gold answer, 也不会直接告诉模型正确下一跳是什么。因此, 模型收到反馈后, 仍然必须自己完成图上的状态更新:

$$\hat{v}_{i+1} \in N^+(\hat{v}_i).$$

也就是说, 修复一条非法路径所需要的能力, 正是模型最初失败的能力: 从当前节点检索合法后继, 并维持更新后的当前状态。因此, verifier-retry 的饱和不是偶然现象, 而是底座模型状态追踪能力不足的直接结果。

这也解释了模型间差异。DeepSeek-V4-Flash 的初始 pass@1 已经接近一半, 说明它本身具备一定图跟随能力, 验证器反馈可以修复其中一部分可纠正错误。Qwen Max 的 pass@1 很低, 即使多次收到结构性反馈, 也经常无法重建合法路径。这说明 verifier-retry 更像是放大已有能力, 而不是创造新的图推理能力。

因此, verifier-retry 的正确定位不是:

verifier teaches the model how to reason over graphs.

而是:

verifier exposes structural errors and gives the model another chance to use whatever graph-following ability it already has.

这一点也解释了为什么 verifier-retry 可以在较低延迟下恢复大量路径合法性, 却不能完全替代 thinking。Thinking 提供的是更高内部推理预算; verifier-retry 提供的是外部错误检测和重复尝试机会。二者缓解失败的方式不同, 成本和上限也不同。

6.3 FailureHop reveals structural bottlenecks

FailureHop 分析进一步说明, retry 后的剩余失败不是随机噪声。若模型每次 retry 都错在不同位置, 那么失败可能主要来自随机解码波动; 但若模型反复卡在同一个 hop, 则说明该位置存在稳定结构瓶颈。

B3. 按数据集 × Prompt 汇总: GFI vs 非法路径差距



在 branching-12hop + verifier-retry 的最终失败样本中, DeepSeek-V4-Flash 约 69.0% 的失败具有 repeated_same_hop=True; Qwen Max 这一比例达到 83.7%。这意味着, 在大量失败样本中, 模型不是没有收到错误反馈, 而是在收到反馈后仍然无法越过同一个状态转移位置。

repeated same-hop rate_{Flash} = 69.0

repeated same-hop rate_{Owen} = 83.7

这个指标比单纯的 pass@K 更能说明 verifier-retry 的边界。pass@K 告诉我们有多少样本最终被修复; repeated same-hop 告诉我们剩余失败为什么修不回来。它说明模型在某些样本上反复失败于同一结构位置, 而不是随机生成了不同错误。

从 FailureHop 分布看, DeepSeek-V4-Flash 的剩余失败主要集中在 hop 2、hop 3 和 hop 12。hop 2/3 通常对应早期分叉点, 说明模型在进入分叉结构后容易选错分支; hop 12 对应终端转移, 说明模型在最后落点时容易为了抵达某个答案节点而生成图中不存在的末步边。此外, DeepSeek-V4-Flash 还有一部分 path too short 错误, 即模型在走满 (k) 跳前提前停止。

这里需要区分两种结尾附近的失败。`path_too_short` 是路径未走满，模型提前终止；`terminal-transition bottleneck` 则是模型走到最后一步附近，但为了抵达某个答案节点而生成非法边。前者是不足，后者是强凑。二者都发生在路径完成阶段附近，但机制不同。

相比之下, Qwen Max 的错误更分散, 多个中间 hop 和终端 hop 都会出现非法边。这说明 Qwen Max 的问题不是某一个局部分叉点特别难, 而是整体多跳状态追踪能力较弱。它通常能够生成长度接近要求、答案与路径末节点一致的轨迹, 但无法稳定保证每一步都沿输入图边移动。

因此, FailureHop 不只是错误统计, 而是一种能力画像。它可以区分:

- **errors concentrated at a few hops:** Interpretation: 局部结构瓶颈, 例如早期分叉或终端转移
- **errors spread across many hops:** Interpretation: 整体状态追踪能力不足
- **high repeated_same_hop:** Interpretation: feedback 后仍反复卡在同一结构转移处
- **path too short:** Interpretation: 无法维持完整 (k)-hop 轨迹

- **errors spread across many hops:** Interpretation: 整体状态追踪能力不足

- **high repeated same hop:** Interpretation: feedback 后仍反复卡在同一结构转移处

- **path too short:** Interpretation: 无法维持完整 (k)-hop 轨迹

这说明 verifier-retry 不只是一个修复机制，也是一个诊断工具。它揭示模型错误是否能在外部反馈下被修正，还是会反复卡在同一类结构障碍上。

6.4 Summary

本节解释了第 5 节结果背后的机制。

第一，结构化输出能提高格式服从和 trace-answer consistency，但不能保证路径逐边合法。模型可以生成答案与路径末节点一致的轨迹，却仍然违反输入图的边约束。

第二，verifier-retry 的提升会饱和，因为验证器只能发现错误，不能替代模型修复错误所需的状态更新能力。修复非法边本质上仍要求模型完成：

$$\hat{v}_{i+1} \in N^+(\hat{v}_i),$$

而这正是失败模型所缺乏的能力。

第三，FailureHop 和 repeated same-hop failure 表明剩余错误是结构性的，而不是随机噪声。大量最终失败会反复卡在同一个 hop，说明模型在收到结构性反馈后仍难以越过同一状态转移瓶颈。

因此，图推理评测不应只检查最终答案，也不应只检查推理轨迹是否与答案一致。对于路径类图任务，真正关键的是验证模型生成的每一步状态转移是否忠实于输入图结构。

7 Analysis

本节进一步分析第 6 节结果背后的机制。第 6 节已经报告了主要结果：弱提示下 Raw GIS 会被位置线索虚高，结构化路径输出能缓解链式图上的 endpoint shortcut，但在困难分叉图上暴露出更深的路径非法问题。本节不再重复完整表格，而是回答三个机制性问题：

1. 为什么 strict JSON-CoT 会产生“自洽但非法”的轨迹？
2. 为什么 verifier-retry 能提升 pass@K，却会明显饱和？
3. 为什么剩余失败不是随机噪声，而是稳定结构瓶颈？

总体而言，分析表明，模型的核心失败并不是“不知道要输出路径”，也不是简单格式错误；更准确地说，是模型无法稳定维护图上的当前状态，并在每一步选择真实存在的后继节点。

7.1 结构化轨迹可能是图任务中的事后合理化

在 branching-12hop + jsoncot_strict 设置下，多数模型的 TAC 很高。这说明模型通常能够生成结构化 JSON，并让 answer 字段等于 path 的最后一个节点。换言之，模型学会了遵守输出接口：

$$\hat{a} = \hat{v}_k.$$

但第 6 节显示，高 TAC 并不保证路径合法。模型虽然能让答案与路径末节点一致，却经常在中间某一步生成图中不存在的边：

$$(\hat{v}_i, \hat{v}_{i+1}) \notin E.$$

这说明 strict JSON-CoT 主要提升了两个能力：格式服从能力和局部文本自洽能力。模型知道要输出 path 和 answer，也知道 answer 应该等于 path[-1]。但是，路径合法性要求的不只是这种文本自洽，而是逐步图状态更新：

$$\hat{v}_{i+1} \in N^+(\hat{v}_i),$$

其中 $(N^+(\hat{v}_i))$ 表示当前节点 $\hat{v}_i$ 的出邻居集合。模型必须在每一步维护“当前节点是谁”，再从当前节点的合法后继中选择下一跳。这个状态更新能力才是路径类图推理的核心。

因此，“自治但非法”的轨迹可以理解为图任务中的结构化事后合理化。在普通自然语言 CoT 中，模型可能先生成答案，再补一段看似支持该答案的解释；在图路径任务中，模型也可能先锁定某个答案节点，再补一条看似通向该节点的路径。区别在于，这里的解释被包装成了结构化路径，因此看起来更可信，但它仍可能不忠实于输入图。

这也是为什么仅检查 trace-answer consistency 不够。TAC 只能证明模型在输出层面没有自相矛盾，不能证明路径中的每一条边都真实存在。对于图路径任务，真正关键的是逐边验证：

$$\forall i, ; (\hat{v}_i, \hat{v}_{i+1}) \in E.$$

因此，PathValid 和 FailureHop 不是附加指标，而是区分“结构化解释”与“图上合法路径”的必要诊断。

7.2 Verifier-retry 有效但会饱和，因为修复仍依赖状态更新能力

Verifier-retry 的作用是把模型输出交给符号验证器检查。如果路径非法，验证器会返回结构性错误反馈，例如哪一步边不存在、路径长度是否错误、起点是否错误、答案是否与路径末节点不一致。第 6 节显示，verifier-retry 确实能提高 pass@K：DeepSeek-V4-Flash 从 pass@1 的 48.6% 提升到 pass@5 的 70.6%，Qwen Max 从 8.4% 提升到 33.4%。

这说明外部符号反馈是有用的。但提升会明显饱和，说明 verifier-retry 区分了两种能力：

- **发现错误**：谁提供：符号验证器；含义：判断路径哪一步不合法
- **修复错误**：谁提供：语言模型本身；含义：根据图结构重新生成合法路径

验证器只能提供第一种能力。它能指出：

Your path is invalid at hop 3:
edge (J9E9, W6S2) does not exist in the graph.

但它不会泄露 gold answer，也不会直接告诉模型正确下一跳是什么。模型收到反馈后，仍然必须自己完成第 7.1 节中的核心操作：

$$\hat{v}_{i+1} \in N^+(\hat{v}_i).$$

也就是说，修复一条非法路径所需要的能力，正是模型最初失败的能力：从当前节点检索合法后继，并维护更新后的当前状态。因此，verifier-retry 的饱和不是偶然现象，而是底座状态更新能力不足的直接结果。

这也解释了为什么 DeepSeek-V4-Flash 和 Qwen Max 的 retry 收益不同。DeepSeek-V4-Flash 的 pass@1 已经接近 50%，说明它本身具备一定图跟随能力，因此反馈可以把部分失败样本修回来。Qwen Max 的 pass@1 只有 8.4%，说明其初始状态追踪能力很弱；即使验证器多次指出错误，模型也不一定能重建出一条合法路径。

所以 verifier-retry 的正确定位不是“创造图推理能力”，而是：

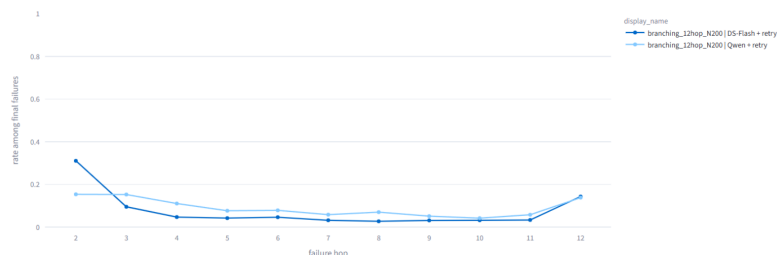
放大已有图跟随能力，并修复其中一部分可纠正错误。

它能把模型从“偶尔走对”推向“更稳定地走对”，但如果模型无法维护图状态，反馈本身并不能替代这种能力。

7.3 重复同跳失败揭示稳定结构瓶颈

final_failure_hop 趋势

错误主要发生在哪一跳



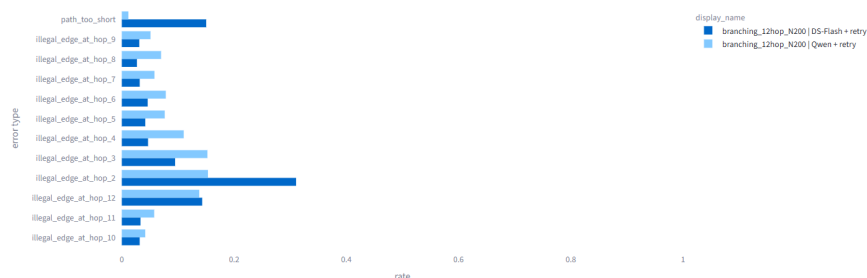
FailureHop 分析进一步说明，verifier-retry 后的剩余失败不是随机噪声。若模型在多次 retry 中每次都错在不同位置，那么失败可能主要来自随机解码波动；但如果模型反复卡在同一个 hop，说明它在某个结构转移位置存在稳定瓶颈。

这一部分不只看错误类型，还看错误发生在哪一跳，方便观察趋势。

错误类型分布 失败 hop 趋势 失败 hop 热力图 精简表

final_error_type 分布

不同模型的最终错误类型分布



实验中，DeepSeek-V4-Flash 约 69.0% 的最终失败具有 repeated_same_hop=True；Qwen Max 这一比例达到 83.7%。这意味着，在相当多失败样本中，模型收到验证器反馈后并没有随机换错，而是反复卡在同一个结构障碍处。

$$\text{repeated same-hop rate}_{\text{Flash}} = 69.0$$

$$\text{repeated same-hop rate}_{\text{Qwen}} = 83.7$$

这两个数字比单纯的 pass@K 更能说明 verifier-retry 的边界。pass@K 只能告诉我们最终有多少样本被修复；repeated same-hop 则告诉我们，剩余失败为什么修不回来。模型不是没有被告知错误，而是在被告知错误后仍然无法越过同一个状态转移瓶颈。

从 FailureHop 分布看，DeepSeek-V4-Flash 的剩余失败主要集中在 hop 2、hop 3 和 hop 12。hop 2/3 通常对应早期分叉点，说明模型在进入分叉结构后容易选错分支；hop 12 对应最后一步终点落点，说明模型在提交最终答案节点时容易生成图中不存在的末步转移。此外，DeepSeek-V4-Flash 还有约 15.0% 的最终失败属于 path_too_short，即模型在走满 (k) 跳之前提前停止，未能生成完整路径。

这里需要区分两类“结尾失败”。path_too_short 是过早终止，模型没有走满规定跳数；terminal-transition bottleneck 则是模型走到了最后一步，但为了抵达某个答案节点而生成非法边。前者是不足，后者是强凑。二者都发生在路径完成阶段附近，但机制相反。

相比之下，Qwen Max 的失败分布更分散，hop 2、3、4、6、12 等位置均有明显非法边错误。这说明 Qwen Max 并不是卡在某一个局部分叉点，而是整体多跳状态追踪能力较弱。它通常能够生成长度接近要求、答案与路径末节点一致的轨迹，但无法稳定保证每一步都沿图边移动。

因此，FailureHop 不只是错误统计，而是模型能力画像。它帮助区分两类失败：

- **错误集中在少数 hop**：机制解释：局部结构瓶颈，例如早期分叉或终端转移
- **错误广泛分布**：机制解释：整体状态追踪能力不足
- **repeated_same_hop 高**：机制解释：模型在反馈后仍反复卡在单一结构转移处
- **path_too_short**：机制解释：模型无法维持完整 (k)-hop 轨迹

这说明 verifier-retry 不仅是纠错机制，也是诊断工具。它揭示了模型错误是否可被反馈修复，还是会在同一个结构位置反复失败。

7.4 位置锚定是答案层失败，不是全部故事

第 6 节的 chain 实验显示，位置锚定确实是图推理中的重要失败模式。在 direct_minimal 提示下，模型可能依赖 endpoint 位置线索，使 Raw GIS 高估真实图跟随能力。这类失败属于答案层假忠实性：模型答案看似随图干预改变，但这种变化可能只是因为 endpoint 的文本位置也随之改变。

然而，位置锚定不是全部故事。在 branching-12hop + jsoncot_strict 中，decoy-last EAR 不再是主导指标，GFI 也不再是最主要证据。此时模型通常能生成结构化路径，并让答案与路径末节点一致；真正的问题变成路径中间边不合法。也就是说，失败机制从答案层位置捷径切换成路径层结构非法。

两类失败可以这样区分：

- **答案层**：失败模式：答案随图变化，但由 endpoint 位置驱动；主要诊断：Raw GIS、PC-GIS、GFI、decoy-last EAR
- **路径层**：失败模式：路径与答案一致，但路径在图上非法；主要诊断：TAC、PathValid、($\Delta_{\{\text{illegal}\}}$)、FailureHop

这一区分正是 GIF 的核心。只看答案层，会漏掉 branching 图上的非法路径；只看路径层，又会忽略 chain 图上的位置虚胖。GIF 同时引入位置控制和路径合法性验证，才能区分两类假忠实性。

这也解释了为什么本文需要同时包含 chain graphs 和 branching graphs。Chain graphs 是检测 endpoint-position shortcut 的高灵敏度设置；branching graphs 则是检测长程状态追踪失败和非法路径生成的高灵敏度设置。二者不是重复实验，而是对应不同失败层级。

7.5 对图推理评测的启示

以上分析对 LLM 图推理评测有三个直接启示。

第一，最终答案准确率不足以证明图忠实性。模型可能答对，但路径非法；也可能在 endpoint-last 条件下答对，却在位置控制后失败。因此，accuracy 只能说明模型是否输出了正确节点，不能说明它是否真正沿图走。

第二，Raw graph-intervention sensitivity 也不是充分证据。答案随图干预改变只是必要条件，不是充分条件。如果正确答案的位置随图干预一起移动，模型可以利用位置线索获得较高 Raw GIS。因此，图干预必须结合位置控制，才能区分真实结构控制和序列化捷径。

第三，结构化 CoT 或 path 字段也不足以证明图忠实性。模型输出了路径，并让路径终点等于答案，并不意味着路径中每条边都存在。对于路径类图任务，真正关键的是逐边验证：

$$\forall i, ; (\hat{v}_i, \hat{v}_{i+1}) \in E.$$

这对现有图增强 LLM 方法也有直接启示。许多方法将 path 作为检索证据、解释、工具调用轨迹或训练奖励的载体。如果不检查路径在输入图上的合法性，就无法排除 “path appears, but is not followed” 的风险。路径出现在 prompt 或 output 中，不等于模型因果地遵循了这条路径。

因此，可靠的图推理评测至少应同时报告三类信息：

1. **位置控制后的答案稳定性**：模型是否在不同序列化下仍输出正确答案；
2. **路径级结构合法性**：模型生成的每一步是否对应输入图中的真实边；
3. **失败位置与错误类型**：模型在哪一跳、以何种方式偏离图结构。

GIF 正是围绕这三类信息构造的诊断框架。它不是为了给模型打一个单一分数，而是为了区分不同类型的图推理假阳性：答案层的位置虚胖，以及路径层的自治但非法轨迹。

7.6 Summary

本节进一步解释了第 6 节结果背后的机制。

第一，strict JSON-CoT 提高了格式服从和 trace-answer consistency，但不能保证路径逐边合法。模型能够生成答案与轨迹末节点一致的输出，却仍可能在中间步骤违反图边约束。

第二，verifier-retry 的提升会饱和，因为验证器只能发现错误，不能替代模型修复错误所需的状态更新能力。修复非法边本质上仍要求模型完成：

$$\hat{v}_{i+1} \in N^+(\hat{v}_i),$$

而这正是失败模型所缺乏的能力。

第三，repeated same-hop failure 表明剩余错误是结构性的，而不是随机噪声。DeepSeek-V4-Flash 和 Qwen Max 中大量最终失败会反复卡在同一个 hop，说明模型在收到结构反馈后仍难以越过同一状态转移瓶颈。

因此，图推理评估不应只检查最终答案，也不应只检查推理轨迹是否与答案一致。对于路径类图任务，真正关键的是验证模型生成的每一步状态转移是否忠实于输入图结构

8 Ablation Study

本节通过消融实验分析 GIF 中不同组件的作用。第 6 节报告了主要结果，第 7 节分析了失败机制；本节进一步回答一个问题：这些发现是否依赖某个特定 prompt、拓扑、推理预算或 retry 设置？

关注四类消融：提示接口、图拓扑与路径长度、内部推理预算，以及 verifier-retry 的重试预算。总体结果表明，本文的核心发现并不依赖单一设置：弱提示下 Raw GIS 容易虚胖，结构化路径输出可以缓解位置捷径，但在长程分叉图上仍会暴露“自治但非法”的路径；thinking 模式可以近乎解决该任务，而 verifier-retry 只能以较低成本部分修复非法路径

8.1 Prompt-interface ablation

[Diagram: mermaid chart omitted in PDF print copy]

首先比较不同 prompt interface 的影响。本文使用三类主要提示：direct_minimal、jsoncot_basic 和 jsoncot_strict。其中，direct_minimal 只要求模型输出最终答案；jsoncot_basic 要求输出路径和答案；jsoncot_strict 进一步要求路径长度、起点、逐边合法性和 answer-path consistency

在 chain graphs 上，prompt interface 对位置捷径有显著影响。以 chain-4hop 为例，direct_minimal 下模型容易产生较高 Raw GIS 但 PC-GIS 为 0，说明答案看似随图变化，但这种变化无法通过位置控制检验。DeepSeek-V4-Flash 在该设置下 Raw GIS 为 68.5%，PC-GIS 为 0%，GFI 为 68.5%；Qwen Max 的 Raw GIS 为 38.0%，PC-GIS 同样为 0%

当提示切换为 jsoncot_strict 后，chain-4hop 基本被解决。DeepSeek-V4-Flash 达到 100% Accuracy、100% PC-GIS 和 100% PathGoldExact；Qwen Max 也达到 99.8% Accuracy 和 99.8% PathGoldExact。这说明结构化路径输出能够显著压制简单链式图上的 endpoint-position shortcut。

然而, prompt interface 的作用有明显边界。在 branching-12hop 上, jsoncot_strict 虽然使 TAC 接近 1, 但并不能保证路径合法。Qwen Max 的 TAC 达到 99.2%, 但 PathGoldExact 只有 8.4%; DeepSeek-V4-Flash 的 TAC 为 99.3%, PathGoldExact 约为 48.5%。这说明 strict prompt 可以提升格式服从和局部自治性, 却不能单独保证图上的逐边状态更新

因此, prompt 消融支持两个结论。第一, 结构化路径输出确实能缓解 answer-only 条件下的位置捷径。第二, 结构化路径输出不是图忠实性的充分条件; 在更难的分叉任务上, 模型仍可能生成自治但非法的路

Ablation finding. Removing structured path output exposes answer-level endpoint shortcuts; adding strict JSON-CoT removes many chain-position shortcuts but does not eliminate trace-level graph illegality on hard branching graphs.

8.2 Topology and depth ablation

[Diagram: mermaid chart omitted in PDF print copy]

接下来, 我们分析图拓扑和路径长度的影响。Chain graphs、branching-4hop 和 branching-12hop 分别对应三个难度层级。

Chain graphs 的主要作用是暴露答案层位置锚定。在这类图中, 从起点到终点的路径结构简单, 模型若真正沿图推理, 应当不同位置序列化下稳定输出同一答案。因此, chain graphs 更适合诊断 Raw GIS 是否被 endpoint position 虚高。

Branching graphs 则引入了状态追踪难度。模型不能只记住一个显著节点, 而必须在每一步根据当前节点选择合法出边。特别是在 branching-12hop 中, 模型需要连续维护较长的当前状态; 局部启发式或简单的位置线索不足以完成任务。

这一拓扑消融解释了为什么 chain-4hop 上 jsoncot-strict 几乎满分, 而 branching-12hop 上仍出现严重 TAC-PathValid gap。两者的差异不只是 hop 数变长, 而是任务从“沿单一路径读出终点”变成“在多分支结构中持续更新状态”。因此, branching-12hop 更能检验模型是否真正执行了图上的多跳转转移。

从指标上看, chain graphs 主要暴露:

$$\text{Raw GIS} - \text{PC-GIS}$$

即答案层图跟随虚胖; branching graphs 主要暴露:

$$\Delta_{\text{illegal}} = \text{TAC} - \text{PathValid}$$

即路径层自治但非法。二者对应不同失败层级, 不能互相替代。

[Diagram: mermaid chart omitted in PDF print copy]

因此, 拓扑和深度消融说明: GIF 需要同时包含简单链式图和困难分叉图。前者用于检测 endpoint anchoring, 后者用于检测状态追踪失败和非法路径生成。

Ablation finding. Chain graphs expose answer-level positional shortcuts, whereas branching graphs expose trace-level state-tracking failures. The main TAC-PathValid gap emerges most clearly in long branching graphs.

8.3 Reasoning-budget ablation

我们进一步消融内部推理预算, 比较 DeepSeek-V4-Pro 的 no-thinking 与 thinking 模式。在相同的 branching-12hop + jsoncot-strict 设置下, no-thinking 模式仍存在明显路径非法问题: Accuracy 约为 56.4%, PathGoldExact 约为 55.4%,

$$\Delta_{\text{illegal}}$$

约为 44.6%。这说明, 即使模型较强, 在没有额外推理预算时仍难以稳定完成长程分叉路径追踪。

相比之下，DeepSeek-V4-Pro thinking 几乎完全解决该任务。其 Accuracy 约为 99.9%，PC-GIS 约为 99.5%，Path-GoldExact 约为 99.9%，

$$\Delta_{\text{illegal}}$$

接近 0。这一结果有两个作用。

[Diagram: mermaid chart omitted in PDF print copy]

第一，它证明 branching-12hop 不是不可解任务。若 thinking 模式可以近乎满分，那么 no-thinking 模式下的失败不应归因于数据构造错误或评估器误判，而应归因于模型在有限推理预算下的状态追踪不足。

第二，它说明非法路径问题可以被更强推理预算显著缓解。换言之，TAC-PathValid gap 不是一种不可避免的输出格式问题，而是与模型搜索能力、状态维护能力和推理预算密切相关。

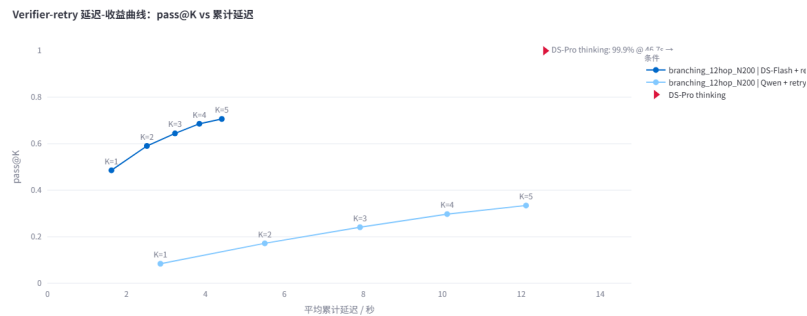
不过，thinking 模式带来明显延迟成本。DeepSeek-V4-Pro thinking 的平均延迟约为 46.7 秒，显著高于 no-thinking 或 verifier-retry 设置。因此，thinking 是高性能高成本方案，而不是低成本修复机制。

Ablation finding. Increasing internal reasoning budget nearly eliminates trace-level illegality, showing that the task is solvable; however, this improvement comes at substantially higher latency.

8.4 Verifier-retry budget ablation

最后，分析 verifier-retry 的重试预算 (K)。在该设置中，模型每次输出路径后由符号验证器检查；若路径非法，验证器返回结构性错误反馈，但不泄露 gold answer。我们比较

$$K = 1, \dots, 5$$



下的 pass@K 和累计延迟。

DeepSeek-V4-Flash 的 pass@K 从 pass@1 的约 48.6% 提升到 pass@5 的约 70.6%。对应平均累计延迟从约 1.62 秒增加到约 4.42 秒。Qwen Max 也从 pass@1 的约 8.4% 提升到 pass@5 的约 33.4%，但其 K=5 延迟达到约 12.12 秒。

- **DeepSeek-V4-Flash:** pass@1: 48.6%; pass@2: 59.0%; pass@3: 64.4%; pass@4: 68.6%; pass@5: 70.6%; latency@5: 4.42s
- **Qwen Max:** pass@1: 8.4%; pass@2: 17.2%; pass@3: 24.1%; pass@4: 29.8%; pass@5: 33.4%; latency@5: 12.12s

这一消融显示，verifier-retry 有明确收益，但收益随 K 增大而递减。前几次 retry 能修复一部分可纠正错误，但到 K=5 后仍有大量样本失败。这说明验证器可以发现错误并提供反馈，但最终能否修复仍取决于模型自身是否具备足够图跟随能力。

不同模型之间的差异进一步支持这一点。DeepSeek-V4-Flash 的 pass@1 较高，说明它已有一定基础图跟随能力，因此 retry 可以进一步放大该能力。Qwen Max 的 pass@1 很低，即使 retry 到 K=5，pass@5 仍明显低于 DeepSeek-V4-Flash。这说明 verifier-retry 不是外部注入图推理能力，而是把模型已有能力从“偶尔成功”提升为“更稳定成功”。

从成本角度看，DeepSeek-V4-Flash 的 verifier-retry 是较高性价比方案：K=5 延迟约 4.42 秒，但获得了约 22 个百分点的 pass@K 提升。Qwen Max 的 retry 成本更高，收益却较低，说明 verifier-retry 的性价比强烈依赖底座模型。

[Diagram: mermaid chart omitted in PDF print copy]

Ablation finding. Increasing retry budget improves pass@K but saturates before reaching the thinking baseline. Verifier-retry amplifies existing graph-following ability rather than creating it from scratch.

8.5 Summary of ablations

综合以上消融, GIF 的主要结论在不同设置下保持一致。

首先, 去掉结构化路径输出后, 模型容易在 chain graphs 上依赖 endpoint-position shortcut, 导致 Raw GIS 高估图跟随能力。加入 strict JSON-CoT 后, 简单链式图上的位置捷径显著减少, 但困难分叉图仍暴露路径非法问题。

第二, 改变拓扑和路径长度会改变失败类型。Chain graphs 主要诊断 answer-level positional anchoring; branching graphs 主要诊断 trace-level state-tracking failure。长程分叉图上的 TAC-PathValid gap 是最稳定、最有诊断价值的失败信号。

第三, 增加内部 thinking 预算几乎可以解决 branching-12hop, 说明任务本身可解, 也说明 no-thinking 模式下的失败来自有限推理预算和状态追踪不足。

第四, 增加 verifier-retry 预算可以部分修复非法路径, 但提升会饱和, 并且强烈依赖底座模型能力。Verifier-retry 是低成本纠错机制, 不是完整替代 thinking 的方案。

因此, 消融实验支持本文的核心主张: LLM 图推理的忠实性不能由单一指标判断。最终答案、Raw GIS、结构化路径输出和 verifier pass@K 分别揭示不同侧面的能力; 只有结合位置控制、路径合法性验证和失败位置分析, 才能区分真正的图跟随与两类假忠实性:

answer-level false faithfulness

和:

trace-level false faithfulness.

9 Conclusion

本文提出了 Graph-Intervention Faithfulness (GIF), 一个用于诊断大语言模型是否真正遵循显式图结构的评估框架。与只报告最终答案准确率的图推理评测不同, GIF 关注两个更细的忠实性问题: 第一, 模型答案是否真的受图结构控制, 而不是受序列化位置线索驱动; 第二, 模型生成的结构化路径是否真的是图上的合法路径, 而不只是与最终答案自治。

实验揭示了 LLM 图推理中的两类假忠实性。

第一类是答案层假忠实性: 在弱 answer-only 提示下, 模型可能表现出较高 Raw GIS, 即答案看似随反事实图干预改变, 但这种敏感性在位置控制后消失。链式图实验进一步显示, 不同模型存在递进式失败模式: DeepSeek-V4-Flash 稳定依赖 endpoint-position shortcut, GPT-5.4-mini 表现出中间型位置捷径依赖, 而 Qwen Max 在较长链上逐渐失去图干预敏感性。这说明, Raw GIS alone 不是可靠的图忠实性证据。

第二类是路径层假忠实性: 在 branching-12hop + jsoncot-strict 设置下, 模型通常能够生成格式正确、路径终点与答案一致的输出, 但这些路径经常包含图中不存在的非法边。换言之, trace-answer consistency 并不蕴含 structural path validity。结构化 CoT 可以让模型的输出更容易解析, 也可以缓解简单链式图上的位置捷径, 但它本身并不能保证模型真正沿图逐边推理

进一步地, thinking 与 verifier-retry 实验揭示了图跟随能力、推理预算和外部验证之间的关系。DeepSeek-V4-Pro thinking 几乎完全解决 branching-12hop, 说明该任务本身可解, no-thinking 模式下的失败主要来自有限推理预算和状态追踪能力不足。Verifier-retry 则能显著提高 pass@K, 并降低非法路径错误, 但其收益明显受底座模型限制。它可以放大已有图跟随能力, 却不能凭空创造图推理能力。FailureHop 分析进一步表明, 剩余失败不是随机噪声, 而是常常集中在早期分叉点或最终转移处, 暴露出稳定的结构性瓶颈

这些结果共同说明，可靠的图推理评估不能只检查最终答案，也不能只检查模型是否输出了看似合理的路径。对于路径类图任务，评测必须同时关注位置控制后的答案稳定性、路径逐边合法性，以及失败发生的位置和类型。GIF 正是围绕这一目标构建的：它将答案层位置控制、路径层符号验证和 verifier-retry 失败分析结合起来，从而区分真正的图跟随与两类常见假阳性。

本文仍有若干局限。我们的实验主要基于符号图和路径到达任务，未来可以扩展到更丰富的图任务，例如可达性判断、最短路、连通性、拓扑性质判断和知识图谱推理。同时，本文主要分析文本序列化图上的行为忠实性，并不直接观测模型内部计算过程。未来工作可以将 GIF 与白盒表征分析、工具调用轨迹分析或训练期干预结合起来，进一步研究模型是否在内部形成稳定的图状态表示。

总体而言，本文的核心结论是：LLM 在图任务中“答案随图变”和“解释自治”都不足以证明其真正沿图推理。只有通过反事实图干预、位置控制、路径合法性验证和失败模式分析，才能更可靠地诊断模型是否忠实于输入图结构。

Limitations

本文提出的 GIF 是一个行为诊断框架，而不是一个覆盖所有图推理能力的通用基准。尽管实验结果揭示了答案层位置捷径和路径层非法轨迹两类重要失败模式，仍有若干限制需要说明。

首先，本文主要使用符号图任务，而不是自然语言知识图谱或真实世界图数据。这样做的优点是可以最大限度削弱实体语义、常识联想和答案先验，使模型必须依赖显式输入图结构作答；prior-only 与 candidate-only controls 也显示，在无图条件下模型基本无法猜中答案。然而，符号图也意味着任务分布与真实知识图谱、社交网络、程序依赖图或科学图数据存在差异。真实图通常包含语义丰富的节点和边，模型可能同时利用结构信息与语义知识。因此，本文结论主要说明：在语义先验被控制的条件下，LLM 仍可能出现图忠实性失败；未来工作需要进一步检验这些失败是否同样出现在自然语言知识图谱和真实图应用中。

第二，本文聚焦于有向图上的固定跳数路径到达任务，即从起点 (s) 出发，恰好走 (k) 跳并输出终点。这一任务适合诊断逐步状态更新和路径合法性，因此能够清楚暴露“TAC 高但 PathValid 低”的问题。但图推理还包括许多其他任务，例如可达性判断、最短路径、连通分量、环检测、拓扑排序、图匹配、子图计数和谱性质判断等。不同任务可能诱发不同失败模式。例如，最短路径任务可能更依赖全局搜索，可达性任务可能不要求输出完整路径，图性质判断则可能不具备明确的逐边 trace。因此，GIF 当前版本最适合诊断路径类图推理忠实性，而不是直接覆盖所有图推理任务。

第三，GIF 是行为层面的忠实性诊断，而不是模型内部机制的直接证明。本文通过反事实图对、位置控制序列化、路径合法性验证和 FailureHop 分析，判断模型输出是否符合“忠实图跟随”应有的行为模式。这些指标可以揭示强烈的外部证据，例如 Raw GIS 在位置控制后崩塌，或者 TAC 接近 1 但 PathValid 很低。然而，它们并不直接观察模型内部是否形成了显式图状态表示，也不能证明模型在神经计算过程中具体如何表示节点、边和路径。未来可以将 GIF 与白盒表征分析、attention/activation patching、工具调用轨迹分析或可解释性方法结合，以进一步研究模型内部是否真的维护了图状态。

第四，GFI 的解释有适用边界。GFI 衡量 Raw GIS 与 PC-GIS 之间的差距，适合诊断 Raw GIS 非零时的图跟随虚胖：如果模型在 raw 条件下看似随图改变答案，但在位置控制后失败，则说明原始图干预敏感性可能被 endpoint position shortcut 污染。但是，当 Raw GIS 本身接近 0 时，GFI 不能被解释为“没有位置偏差”或“模型更忠实”。在这种情况下，模型可能只是完全失去图干预敏感性。因此，本文在解释 GFI 时将其限定为 answer-level inflation 的诊断指标，并结合 decoy-last EAR、PC-GIS、PathValid 和 FailureHop 共同分析，而不单独依赖 GFI 作结论。

第五，verifier-retry 是一个受控诊断设置，而不等同于真实部署中的完整推理系统。本文的验证器只返回结构性错误反馈，例如格式错误、路径长度错误、非法边位置或 trace-answer mismatch，并不泄露 gold answer。这一设计避免了 oracle leakage，并允许我们研究外部符号反馈能否帮助模型修复非法路径。然而，实际应用中未必总能获得完整图结构、精确验证器或多次重试预算。此外，verifier-retry 的成功率和延迟强烈依赖底座模型、API 实现、响应速度和输出格式稳定性。因此，本文不主张 verifier-retry 是通用解决方案，而将其作为一种诊断工具：它可以区分“模型能否在反馈下修正错误”和“模型是否根本缺乏图跟随能力”。

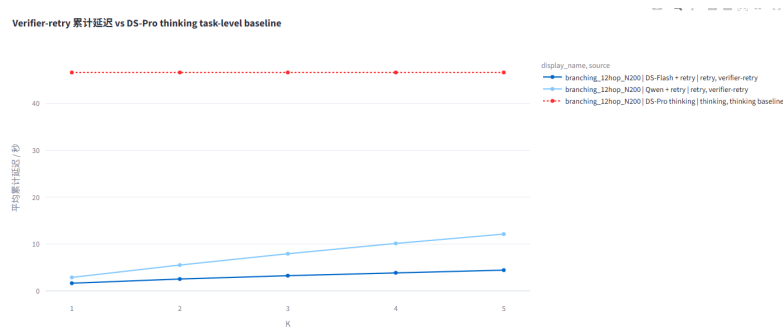
第六，模型覆盖范围仍然有限。本文评估了多个闭源或 API 模型及其不同推理模式，包括 no-thinking、thinking 和 verifier-retry 设置，但没有系统覆盖所有主流闭源模型和开源权重模型。不同模型的训练数据、指令调优方式、上下文长度、工具使用能力和推理模式可能显著影响图任务表现。尤其是 reasoning models、tool-augmented models 和专门图推理模型可能呈现不同的 failure profile。因此，本文结果应被理解为对若干代表性模型的诊断，而不是对所有 LLM 图推理能力的最终结论。

第七，延迟比较存在一定实现依赖。本文报告了 thinking 与 verifier-retry 的平均延迟，并分析其成本—效果关系。但延迟受模型提供方、网络状态、API 调度、输出长度和重试实现影响，不应被视为硬件无关的绝对性能指标。更稳妥的解释是：在相同实验环境和日志记录方式下，thinking 提供高性能但高延迟的基线，verifier-retry 提供低成本但不完全的纠错路径。未来可以在本地部署模型或统一推理后端上复现实验，以获得更严格的计算成本比较。

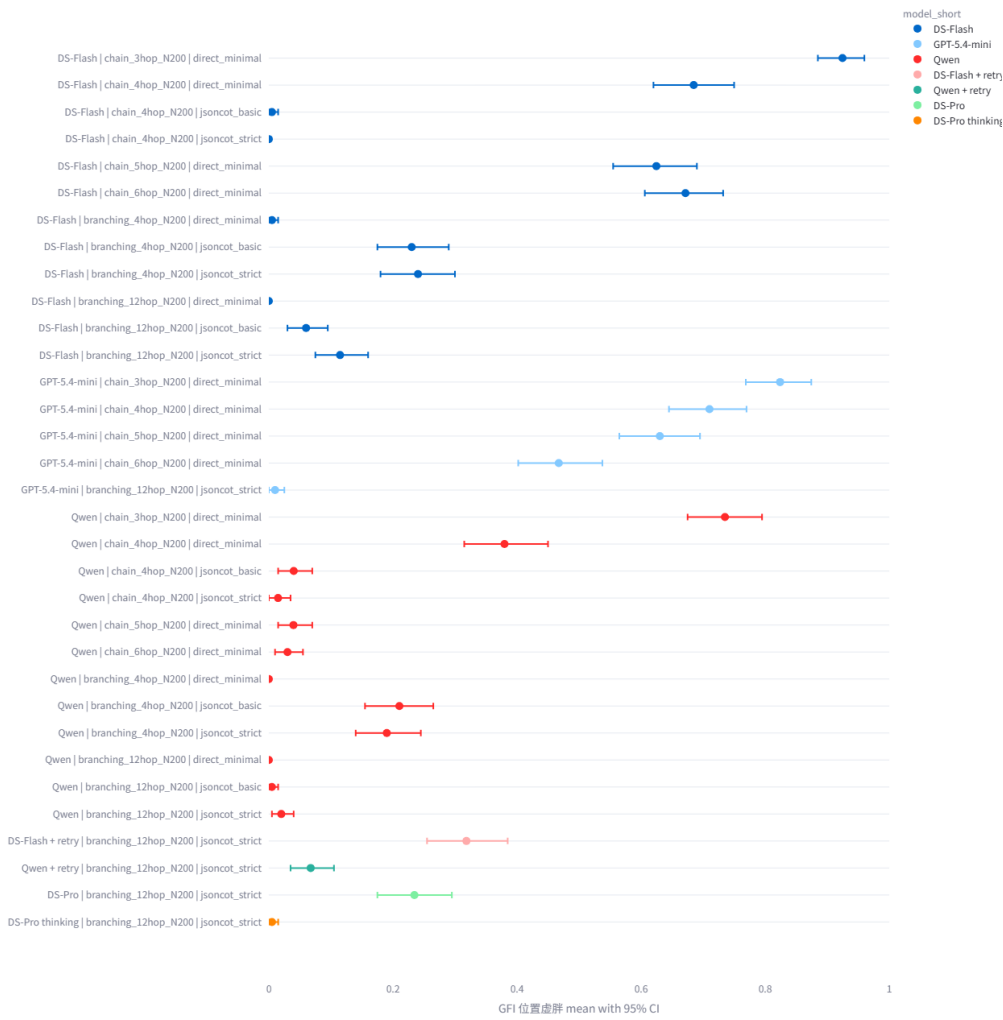
最后，本文的早期 pilot 曾发现极高的 decoy-last EAR，但后续检查发现旧版生成器中边列表未充分打乱，可能高估 endpoint anchoring。正式 main 实验已修正这一问题，对边顺序进行随机化并进行程序校验。因此，本文不依赖早期 pilot 的极端锚定结果，而是基于修正后的 main 实验报告结论。这一过程也说明，图序列化实验对数据生成与顺序控制非常敏感；未来基准应公开生成器、校验脚本和原始日志，以便复现和审计。

综上，这些限制并不削弱本文的核心主张：最终答案正确、Raw GIS 较高或 trace-answer consistency 较高，都不足以证明 LLM 真正忠实于输入图结构。GIF 的贡献在于提供一种分层诊断方式，用位置控制暴露答案层假忠实性，用路径合法性验证暴露轨迹层假忠实性，并用 verifier-retry 与 FailureHop 分析进一步揭示失败是否可修复、是否具有结构性。

附录



Bootstrap 95% CI: GFI 位置虚胖



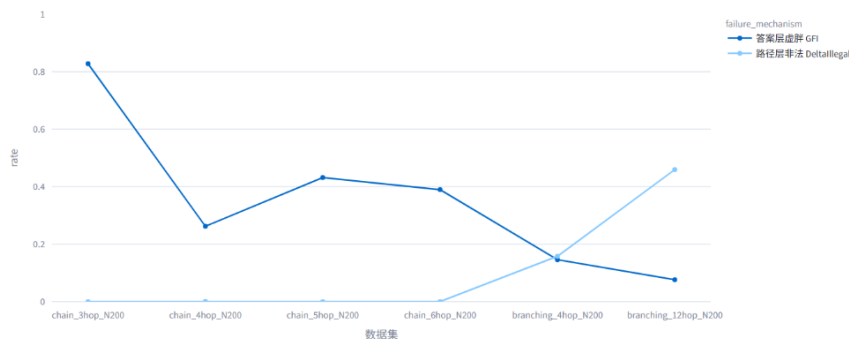
不同模型的最终错误类型分布





B2. 按数据集汇总: GFI vs 非法路径差距

按数据集汇总: 答案层虚胖 vs 路径层非法



□ 第一梯队•定义 GIF 自身 / 直接动机 (4 篇, 最核心)

[1] 图没变, 字变了: LLM 图推理的序列化不变性危机 (Lost in Serialization) —GIF 最近邻, 序列化不变性危机 = 引言第 2–3 段的直接动机。[2]. 图帮文本去噪, 文本帮图复活 (TGS-RAG) —GFI = Raw GIS – PC-GIS 精确表述、pilot 数字、“生成层留空”缺口、真实 RAG testbed。[3]. 别让大模型手算图: 图推理应从复现算法转向设计算法 (Simple-RTC) —“工程绕过 vs 诊断显微镜”的对照, metrics 语境。

□ 第二梯队•忠实性谱系 (Related Work 主战场, 9 篇)

[4]. 思维链会招供, 最终答案会闭嘴 (Lie to Me, Young 2026) —verbalization vs structural faithfulness 的关键区分; 跨家族 roster 借用源。[5]. 思维链不忠实: 模型如何把偏见包装成推理 (Turpin 2023) —行为干预测忠实性的思想祖先。[6]]. 给 CoT 推理拍 X 光: 用计算图验证 (CRV) —把 GIF 定位成黑盒/任务无关诊断框架; pilot 规模小 → 定位为 diagnostic benchmark 的依据。[7]. 忠实性不是“看起来合理”: NLP 解释该如何证明自己没撒谎—faithfulness 的严格定义来源, 写 intro/定义节必引。[8]. 会解释, 不代表真会想: 大模型忠实思维链为何困难 (Hardness of Faithful CoT) —“忠实本身就难”的理论背书。[9]. 大模型的“推理小作文”可信吗? (Measuring Faithfulness in CoT) —忠实性测量方法谱系。[10]. 先有答案, 再编理由: 真实场景中的 CoT 不忠实 (CoT in the Wild) —真实场景不忠实的证据。[11]. 偏见进了答案, 却没进思维链 (Bias & CoT Faithfulness, VLLM) —偏置注入法的旁证。[12]. 模型嘴里的推理, 未必是脑中的理由: CoT 忠实性失真—忠实性失真证据。□ 这篇 frontmatter 的“论文题目”写成了 FiDeLiS, 疑似复制错, 建议核对。

□ 第三梯队•位置偏置/序列化 (支撑 endpoint anchoring, 5 篇)

[13]. 排名别看位置: InvariRank —“注意力泄漏 □ 位置编码”的机制分解; endpoint anchoring 升级到机制归因的钩子。[14]. 排在前面就更香: RISE —位置偏置量化框架; 给你第二条反锚定干预臂 (逐跳单步选择)。[15]. 把图塞进 KV:

Graph-KV —强制串行化 → 位置偏置/Lost-in-the-Middle, 序列化弊端的总论来源。[16]. **选项换个座位, 模型换个答案** (MCQ 选项顺序敏感) —位置敏感的跨任务旁证。[17]. **顺序一换, 模型就乱: LLM 输入排列敏感性**—排列敏感跨 benchmark 证据。□ frontmatter“ 论文题目” 写成 Graph-constrained Reasoning, 明显是复制错, 建议修。

□ 第四梯队·图推理方法 (被诊断的“ 增强路线” + 数据/testbed, 11 篇)

[18]. **图结构不是文本: LLM 需要导航** (GraphWalk) —底座 (合成图生成器) + 反例 + 弹药三合一。[19]. **边看边走: SoG** (Search-on-Graph) —最佳真实世界 testbed。[20]. **把计划画成图: 用 GNN 抓任务规划错接漏步** (GNNVerifier) —真实图数据源 (TaskBench/UltraTool)。[21]. **路径别全塞: KGQA 需要看几跳历史?** (BPC) —行为冗余的层级互补论据。[22]. **让大模型沿着图走: 知识图谱把推理从“ 表演” 拉回“ 路径”** (RoG) —忠实 KG 推理代表方法。[23]. **别让大模型硬算图: 拆成三步调用图工具** (GCR, Graph-constrained Reasoning) —标题自带“ faithful reasoning”, 正面对照。[24]. **别让 CoT 瞎想: 让大模型顺着知识图谱走** (Follow the Path) —KG 路径推理。[25]. **让大模型沿着图走: Graph-CoT** —RAG 从找文本升级到走关系。[26]. **给大模型一张地图: FiDeLiS** —忠实 KGQA 约束路径。[27]. **边想边查图: 用知识图谱逐步补脑** (Graph-Augmented Reasoning) —迭代式 KG 检索。[28]. **KG 当奖励模型: 路径对齐训练组合推理**—下游意义/安全攸关迁移的研究故事 (“忠实诊断是前置必要条件”)。[29]. **LLM 真的会看懂图吗?** (Revisiting Graph Reasoning Ability) —图推理失败的实证弹药。

□ 第五梯队·多跳评测/反事实基准 (3 篇)

[30]. **别让模型背答案: CofCA** —反事实多跳、抗记忆虚胖, 与你“ 反事实图构造” 是姊妹方法。[31]. **Omanic: 一步步检查多跳推理**—step-wise 多跳评测。[32]. **答案对了, 不代表会推理** (多维度推理步评估) —推理质量评测维度。

□ 第六梯队·验证/Agent (弱关联, 按需, 3 篇)

[33]. **把推理拆成图: GoV 用 DAG 验算**—DAG 拓扑序验证骨架。[34]. **把多条思路织成一张图: GraphReason** —图结构验证法。[35]. **从“ 会调用” 到“ 会纠错”: 工具增强模型的元验证与反思学习**—工具增强忠实性, 关联 verify-retry 那条实验线。[36]. **Thinking While Listening** (FSRM) —最弱, 仅“ 递归架构能自发学结构但是否忠实仍需诊断” 一句。